An Optimal Deterministic Adversary for Sorting

Nicholas Nash
ORCID: 0009-0003-0475-7852

8 November 2026

Abstract

This paper describes an optimal *deterministic*, polynomial-time adversary for ordinary sorting. For any correct sorting algorithm, the adversary forces that algorithm to make $(1 - o(1))n \log_2 n$ comparisons. The best previously known adversary forced $\frac{37}{64} n \log_2 n$ comparisons. The adversary recursively moves items into ordered subproblems, extending the classical framework in which items move down a binary tree.

Methodological disclosure

Except for the abstract, the text of this paper is AI-generated using GPT 6 Astra. The author is working on a clear, human-readable exposition.

1 Introduction

We consider comparison sorting of n distinct labelled keys. A sorter chooses each pair using the answers to its earlier comparisons and must be correct for every strict total order on the labels. A sorting adversary supplies those answers as the comparisons arrive. It is *legal* if every finite sequence of its answers is consistent with at least one total order. The adversary knows n and sees the comparisons, but need not inspect the sorter that generates them. Throughout, log denotes $\log_2$, and ln the natural logarithm; we assume $n \geq 2$.

The classical sorting lower bound guarantees the existence of a hard input for every correct sorter. Here the problem is to produce hard answers by one *deterministic, polynomial-time* procedure. The polynomial bound applies to the adversary's computation; the comparison lower bound applies to the sorter. Without the time restriction, a deterministic optimal adversary is immediate. With randomness, polynomial-time adversaries achieve the optimal asymptotic bound in expectation and with high probability.

Without the time restriction. Keep all total orders consistent with the answers so far. Each comparison partitions them into two classes; answer so as to retain a larger class. After q comparisons, at least $n!/2^q$ orders remain, forcing $q \geq \log(n!)$. This rule is deterministic, but enumerating its classes may require factorial work. A consistent total order is a *linear extension* of the partial order given by the answered comparisons; counting these extensions exactly is #P-complete [8].

Deterministic adversaries from graph entropy. Kahn and Kim [10] already give a deterministic polynomial-time adversary for sorting with partial information. Starting from any given partial order P, it answers comparisons online and forces $\Omega(\log e(P))$ comparisons, where $e(P)$ is the number of linear extensions of P. Their method uses graph entropy and convex

optimization via the ellipsoid method. For ordinary sorting, the initial partial order has no relations, so $e(P) = n!$ and their guarantee is $\Omega(n \log n)$. Thus deterministic polynomial-time adversaries are available even for arbitrary initial partial information. The remaining question for ordinary sorting is whether such an adversary can force $(1 - o(1))n \log n$ comparisons.

With randomness. An efficient adversary can simply choose a uniformly random permutation of the labels, keep it hidden, and answer from the ranks in that permutation. For every fixed correct sorter, the usual comparison-tree argument gives an expected cost of at least $\log(n!)$. It also gives a high-probability bound. At most 2^q permutations can terminate at leaves of depth at most an integer q, so the comparison count C satisfies

$$\mathbb{P}(C \le q) \le \frac{2^q}{n!}.$$

Thus $C \ge \log(n!) - O(\log n)$ with probability $1 - o(1)$. Generating an exactly uniform permutation takes expected polynomial bit time; subsequent answers require only rank lookups. A fixed polynomial time bound is obtained by using an almost uniform permutation, with a vanishing additional failure probability. These guarantees also hold for an always-correct randomized sorter whose random bits are independent of the adversary's: condition on the sorter's random choices and apply the same argument.

For any prescribed permutation, a correct sorter can first check its $n - 1$ consecutive pairs, finish if all checks agree, and otherwise use a general sorting algorithm. That permutation then costs only $n - 1$ comparisons. The deterministic adversary must choose its answers in response to the questions it receives.

Randomized approximation of the number of linear extensions. Randomization also makes it possible to estimate how many linear extensions remain after each possible answer. Given the current partial order P, assign each label a real coordinate in $(0, 1)$ and form the region

$$K(P) = \{X \in (0,1)^n : X_i < X_j \text{ whenever } i <_P j\}.$$

Each allowed total order occupies volume $1/n!$ in the unit cube. Hence $\mathrm{vol}(K(P)) = e(P)/n!$ [7]. The two possible answers to a comparison cut this region into two parts, and their volumes are proportional to their counts of allowed orders.

Dyer, Frieze, and Kannan [9] give a randomized polynomial-time algorithm for relative volume approximation and explicitly note its application to linear extensions. To use it here, first check whether the queried relation is already forced by P. If so, return it directly. Otherwise both directions are feasible and give positive-volume regions. Estimate their volumes and choose the larger estimate. When both estimates have relative error at most ε, the chosen part retains at least $(1 - \varepsilon)/2$ of the current volume. The counting argument then forces

$$q \ge \frac{\log(n!)}{1 - \log(1 - \varepsilon)} = (1 - O(\varepsilon)) \log(n!)$$

comparisons. Taking $\varepsilon = 1/\lceil \log(n + 1) \rceil$ gives $n \log n - O(n)$.

For a high-probability guarantee, allow $Q = 2n \lceil \log(n + 1) \rceil$ queries before fixing a consistent order. Amplify each volume estimate to failure probability at most n^{-4}. Each call uses fresh randomness, so this bound holds conditional on the preceding history. A union bound over at most $2Q$ estimates gives total failure probability $O(\log n / n^3)$, even for adaptive queries. These accuracy requirements preserve polynomial time. Every run remains legal, including when an estimate fails, because only feasible answers are offered.

The issue is therefore *derandomization*: obtain the optimal asymptotic comparison bound with a single deterministic response rule whose work per comparison is polynomial in n. Kaplan, Zamir,

and Zwick [5, Section 7] explicitly ask for an adversary forcing $(1 - o(1))n \log n$ comparisons. Our main theorem answers this question.

Theorem 1.1. *There is a deterministic legal adversary, initialized using only n, that performs a fixed polynomial number of bit operations per comparison and forces every correct comparison sorter to make*

$$(1 - o(1))n \log n$$

comparisons.

Merge sort uses at most $n \lceil \log n \rceil$ comparisons, so this bound is asymptotically optimal. The derandomization uses a restricted recursive representation of the remaining orders. The structure of these regions allows the adversary to calculate the required volume estimates deterministically in polynomial time. Its running-time guarantee is asymptotic and involves very large parameter thresholds.

Section 2 reviews the classical tree adversaries and the construction of Kaplan, Zamir, and Zwick. Section 3 gives an overview of our state, comparison operation, and refinement operation. Section 4 specifies the complete algorithm and its parameters. Section 5 proves that its answers remain consistent with a total order. Section 6 justifies the local calculations. Sections 7 and 8 establish the comparison bound, and Section 9 proves polynomial running time. The geometric and numerical proofs used by the local calculation are in the appendices. The manuscript source is available online.[1]

2 Related Work

2.1 Binary-tree adversaries

Atallah and Kosaraju [1] gave an efficient sorting adversary with a lower bound of $\frac{1}{2}n \log n$ comparisons. Related adversaries occur in the work of Borodin, Guibas, Lynch, and Yao [2] on searching after partial sorting, Richards and Vaidya [3] on the distribution of comparisons among keys, and Brodal, Chaudhuri, and Radhakrishnan [4] on maintaining the minimum. The binary-tree rule of Borodin et al. and Brodal et al. provides the starting point for our construction.

Place every item at the root of an ordered binary tree. Items in disjoint subtrees have the left-to-right order of those subtrees. A comparison changes the placement as follows.

- If both items occupy the same node, move one to its left child and the other to its right child, and answer accordingly.
- If one item is at an ancestor of the other, move the higher item to the child whose subtree does not contain the lower item.
- If their nodes are unrelated by ancestry, answer from their left-to-right order, without moving either item.

For a concrete example, start with A, B, C, D at the root. Answering $A < B$ moves A to the left child and B to the right, while C, D remain at the root. If the sorter next compares A and C, the ancestor rule moves C to the right child and gives $A < C$. Comparing D and B then moves D to the left child and gives $D < B$. The two children now contain $\{A, D\}$ and $\{B, C\}$. A subsequent comparison of D with C is answered $D < C$ without moving either item. The orders within $\{A, D\}$ and within $\{B, C\}$ remain unresolved. Figure 1 shows the placements.

Each comparison increases the sum of item depths by at most two. At completion, occupied nodes must be distinct and unrelated by ancestry: otherwise two represented items can occur in either relative order, leaving an unresolved comparison. Associate the root with $(0, 1)$, and

[1] https://github.com/nicknash/sorting-adversary-proofs/tree/main/for-humans

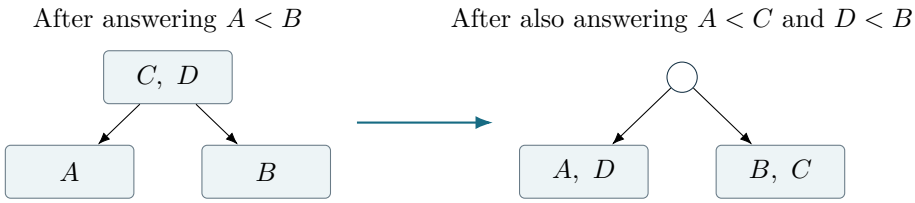


Figure 1: The binary-tree adversary. The first comparison separates two items at the same node. The next two comparisons each move an item from an ancestor into the opposite child. In the final state, comparing D with C requires no movement.

associate each child with the corresponding half of its parent's interval. A node of depth d_i therefore represents an interval of length 2^{-d_i}. At completion these intervals are disjoint, giving

$$\sum_i 2^{-d_i} \le 1, \qquad 2^{-\frac{1}{n}\sum_i d_i} \le \frac{1}{n}\sum_i 2^{-d_i} \le \frac{1}{n}.$$

Convexity gives the bound on $2^{-\frac{1}{n}\sum_i d_i}$. Hence $\sum_i d_i \ge n \log n$, and at least $\frac{1}{2} n \log n$ comparisons are necessary [4, Section 4.1].

The tree representation can settle relations that have never been queried. For example, start again with four items at the root and answer $A < B$ and $C < D$. Placing A, C in the left child and B, D in the right child also commits the adversary to $A < D$ and $C < B$. Neither relation follows from those two answers alone.

2.2 The adversary of Kaplan, Zamir, and Zwick

Kaplan, Zamir, and Zwick [6] replace each step of a four-child tree by a small directed acyclic graph (DAG), allowing an item to move towards a group of consecutive children before selecting one child. Number the children $0, 1, 2, 3$. The intermediate node $[i, j]$ represents the possible destinations $i, i+1, \ldots, j$. The parent itself is $[0, 3]$, and $[i, i]$ is child i. For $i < j$, a downward step removes one endpoint of the range, reaching $[i, j-1]$ or $[i+1, j]$. Figure 2 shows this interval lattice. For example, $[0, 2]$ and $[1, 3]$ both lead to $[1, 2]$; the intermediate nodes have shared descendants. The same construction is repeated at every node of the four-child tree.

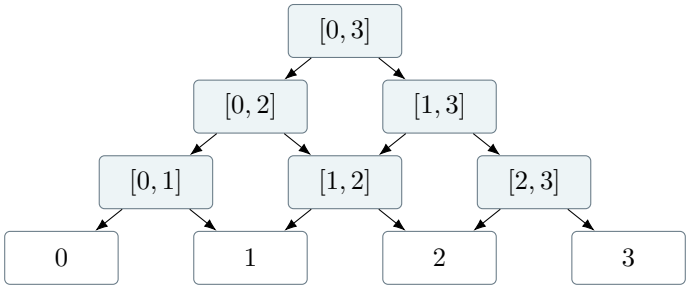


Figure 2: One step of the four-child tree in the adversary of Kaplan, Zamir, and Zwick [6, Section 5]. An intermediate node records a range of possible children. In particular, an item at $[1, 3]$ has not committed to child 1, 2, or 3.

Within a node, the adversary stores unpaired items and disjoint ordered pairs. There can be many items at a node, but its retained comparison components have at most two vertices. Tree nodes and intermediate nodes representing three children may store pairs; intermediate nodes representing two children store only unpaired items. Comparing two unpaired items A, B at a tree node can therefore record $A < B$ as a pair without moving either item.

A comparison between pairs in different nodes illustrates the role of the lattice. Suppose $A < B$ is retained at $[0, 2]$ and $C < D$ at $[1, 3]$. On comparing A and C, one permitted response

is $A < C$: move A into child 0 and B into $[1, 2]$, leaving the pair $C < D$ at $[1, 3]$. Figure 3 shows the change. The old answer $A < B$ is now represented by the items' positions, while $C < D$ remains a retained pair [6, Table 11].

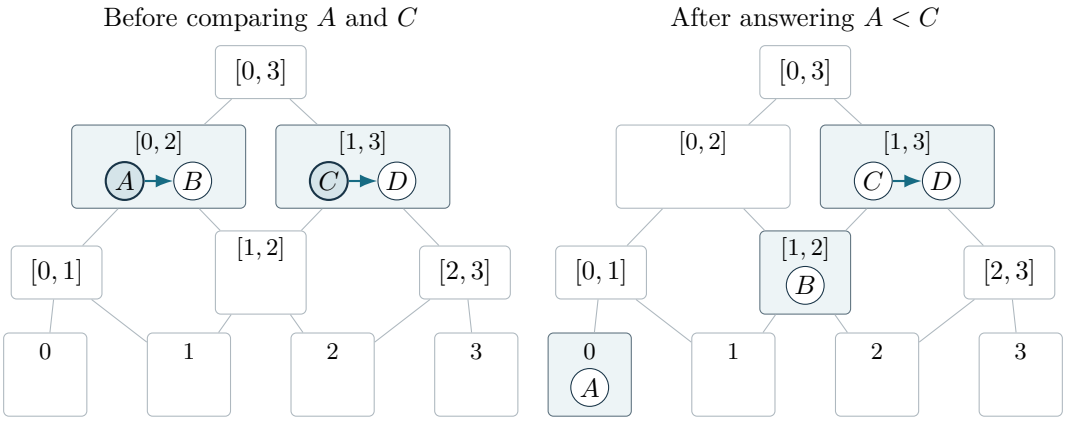


Figure 3: A comparison between pairs at different nodes in the adversary of Kaplan, Zamir, and Zwick [6, Table 11]. Shaded items are queried. Blue arcs are retained comparisons; gray lines are lattice edges. The new answer $A < C$ is enforced by the disjoint child ranges assigned to A and C.

A retained pair also restricts its members' possible destinations: they cannot enter the same child of the underlying four-child tree. For example, if $A < B$ is stored at $[0, 2]$, then A can enter only child 0 or 1, and B can enter only child 1 or 2. The pair's order thus narrows each member's possible destinations, and the adversary uses these restrictions when choosing subsequent comparison answers [6, Section 5].

Their analysis measures progress using item depths, intermediate placements, and retained pairs. The comparison rules bound the increase in this measure and give a lower bound of $\frac{37}{64} n \log n$ comparisons, improving the earlier $\frac{1}{2} n \log n$ bound. Their concluding discussion proposes allowing more complicated comparison components than pairs, and asks whether an efficient deterministic adversary can force $(1 - o(1))n \log n$ comparisons [6, Section 6].

3 Overview of our adversary

We now give an overview of the data structure maintained by our adversary and how it is updated in response to comparison queries from the sorting algorithm.

3.1 The data structure

Our data structure is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG), whose vertices are the items recorded at that node. Figure 4 shows an example; Algorithm 1 gives the comparison procedure.

Each item has a record (x, X_x, a_x, b_x) stored at exactly one node. The label x identifies its key, and X_x is a symbolic variable for its position among the items at that node. The fields a_x, b_x restrict this variable to the open interval $(a_x, b_x) \subseteq (0, 1)$. We call an item *free* at its node while its position remains variable.

In a node's DAG, an arc $x \to y$ records a comparison answered $x < y$ between two items stored there, imposing $X_x < X_y$ on their positions. A feasible assignment of positions at that node satisfies both its interval bounds and its arc inequalities. Each answered comparison between free items contributes at most one arc.

Initially the tree consists of just the root, containing all records with bounds $a_x = 0$ and $b_x = 1$. The position variables X_x are unassigned, and the root's DAG has no arcs. Thus every

assignment in $(0,1)^n$ is feasible, and every total order can be obtained by assigning distinct positions.

Each child is attached to a fixed position $c \in (0,1)$ in its parent, called its *anchor*. For any feasible assignment at the parent, an item with position below c precedes every item in that child's subtree; an item with position above c follows every item in it. Subtrees of different children are ordered by their anchors. All items of each child subtree therefore occur consecutively in the resulting total order. Within the child, position variables again range over $(0,1)$, and its DAG and interval bounds describe the possible orders inside this block.

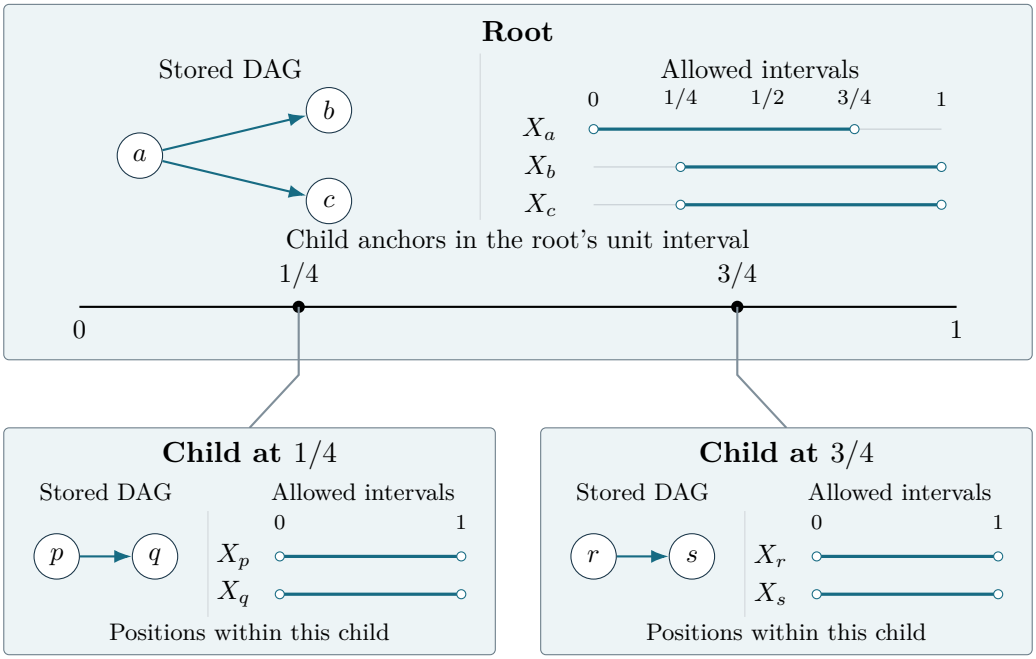


Figure 4: A state of the adversary. Each large box is a tree node, containing a DAG and one open interval for each of its items. Blue segments show the interval bounds; blue arcs impose additional inequalities between positions. Solid dots are fixed child anchors, and gray lines connect them to their children. Vertices are arranged to display the DAG's arcs; the horizontal bars specify their allowed positions.

In the root of Figure 4, the DAG records $a < b$ and $a < c$, while the bars give $X_a \in (0, 3/4)$, $X_b \in (1/4, 1)$, and $X_c \in (1/4, 1)$. The children contain $p < q$ and $r < s$. To read an order from this state, choose feasible positions at each node, avoiding ties with other items or child anchors. List the items and anchors from left to right, and recursively replace each anchor by its child's order. For example, root positions $(X_a, X_b, X_c) = (1/3, 1/2, 7/8)$ give

$$p < q < a < b < r < s < c.$$

Other feasible choices give other orders. Every order represented this way must respect all previous answers to the sorter. Those answers are also kept in a separate graph on the original item labels.

3.2 Moving an item into a child

The operation that moves an item x into a child is called a *refinement*, denoted `Refine`(x). First propagate all interval bounds implied by the DAG. This gives each item's *effective interval*, its full range of feasible positions. Choose a point c inside x's effective interval and fix its parent position at $X_x = c$.

The incident arcs then become interval bounds: $y \to x$ gives $X_y < c$, and $x \to z$ gives $X_z > c$. Remove x and these arcs from the parent, retain the bounds, and propagate them through

the remaining DAG. Insert x into the child anchored at c, creating it if needed. In that child, x receives a fresh interval $(0, 1)$ and no incident arcs; the child's existing constraints remain unchanged. Figure 5 illustrates reuse of an existing child.

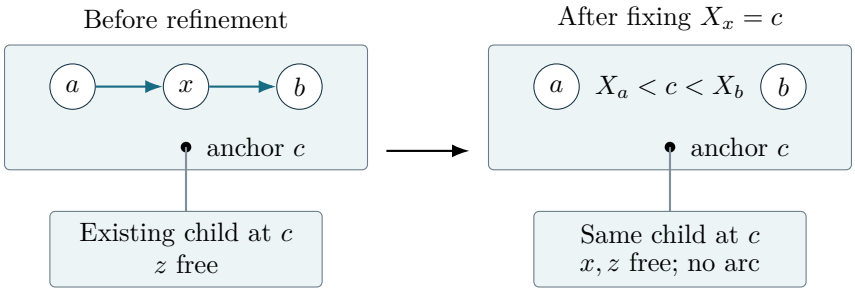


Figure 5: The two arc inequalities become bounds on the remaining parent coordinates. The new coordinate of x orders it within the child, where both $x < z$ and $z < x$ remain possible. The anchor c locates the child block among the parent items; the child's constraints determine the orders within that block.

Thus the parent fixes where the child block occurs, while the order of x within that block stays open. Lemma 5.2 proves that this operation preserves all answered comparisons. The deterministic rule for choosing c is given in Section 6.2; its role is to control how many possible orders refinement discards.

3.3 Answering a comparison

A query on labels A, B is handled at the first node where they are represented differently. Starting at the root, descend while both labels belong to the same child subtree. At the stopping node, each is either a free item or the anchor of a child containing it. In Figure 4, comparing p with q descends into the left child; comparing p with r uses the anchors $1/4$ and $3/4$; and comparing a with p compares X_a with $1/4$ at the root.

Two distinct anchors already determine an answer. Otherwise, the allowed position assignments at this node form a region, cut into two parts by the possible answers. The adversary estimates their relative volumes and chooses the answer with the larger estimate. This keeps at least half the old volume, up to a small error. The estimates are calculated deterministically using only a small neighborhood of the queried items in the DAG, with interval bounds inherited from the full graph (Theorem 6.1).

An answer between two free items adds an arc to their DAG; an answer between a free item and an anchor tightens the item's interval. For the volume estimates to be accurate, degrees and directed paths must stay below prescribed limits, and at least one queried free item must have an effective interval of sufficient width. If all queried free intervals are too narrow, refine one of those items before answering. After answering, we restore the graph bounds by refining vertices with too many neighbors, then refining every vertex of any selected path that is too long. Algorithm 1 summarizes the procedure; all selections use fixed tie-breaking rules.

Algorithm 1: `Compare`(A, B): overview of a comparison query.

```
Input: Distinct item labels A, B and the current adversary state.
Output: An answer A < B or B < A, with the state updated.
while true do
    v ← first node, starting from the root, where the labels' representations differ
    if both labels are represented by child anchors then
        answer ← their left-to-right order
        return answer
    Propagate interval bounds through the DAG at v
    if at least one queried free interval is wide enough then
        break
    Refine(one of the queried free items)
Estimate the volume fraction supporting each answer, using a local DAG
answer ← the answer with the larger estimate
Store the arc or interval bound for answer at v
Propagate interval bounds through the DAG at v
while a vertex at v has too many neighbors do
    Refine(such a vertex)
while the DAG at v has a directed path that is too long do
    Select such a path and Refine all its vertices
return answer
```

Refining a queried item restarts the search from the root because the node that handles the query may change. In the last loop, the whole selected path is refined before another is selected.

The proofs must account for the orders discarded by refinements as well as by comparison answers. Section 5 establishes legality; Sections 7 and 8 bound the total loss and prove the comparison lower bound. Section 9 shows that the local calculations and all updates take polynomial time.

4 The complete algorithm

We first describe what a state means and how an item moves from a parent to its child. We then give the comparison rule and the conditions that trigger these moves. The numerical parameters are collected at the end of the section, after their roles have been explained.

4.1 A tree of groups, each with its own coordinates

The state is a rooted tree. Each node stores item records and a single directed acyclic graph (DAG) on the items stored there. We call this DAG the node's *active graph*.

Each item has a record (x, X_x, a_x, b_x) stored at exactly one node. The label x identifies its key, and the symbolic variable X_x describes its position among the items at that node. The record's bounds impose $a_x < X_x < b_x$, with $(a_x, b_x) \subseteq (0, 1)$. We call the item *free* while its position remains variable. An arc $x \to y$ in the node's active graph records the requirement $X_x < X_y$.

A child occupies a fixed position $c \in (0, 1)$ in its parent. This position, called its *anchor*, determines how the child's items compare with items at the parent: $X_x < c$ places x before every item in the child subtree, and $X_x > c$ places it after every item in that subtree. If two children have anchors $c < d$, every item in the subtree at c precedes every item in the subtree at d. Each child subtree thus occupies a consecutive block in the represented order. Its internal order is determined recursively, using position variables in $(0, 1)$ at the child. Different children of a

node have different anchors. The allowed assignments at each node must have positive volume; Section 5 proves that every operation preserves this property.

To read an order from the state, choose allowed values of the free coordinates at each node. At the root, list its free items and child anchors from left to right. Replace each anchor by the list obtained in its child, and continue recursively. We choose values avoiding ties; these form all but a set of volume zero. Every item appears exactly once in the resulting list. The collection of feasible assignments therefore specifies the family of total orders represented by the state.

Initially the tree has only a root. All n items are free there, with records $(x, X_x, 0, 1)$ and unassigned position variables X_x. There are no arcs, so the feasible assignments form $(0,1)^n$. Assignments with distinct positions represent all $n!$ total orders. The algorithm will ensure that every order it represents is consistent with all the comparisons it has answered. It may discard additional orders when it changes the representation; the lower-bound proof will account for that loss.

4.2 Refinement: fixing the parent position

The basic operation moves an item x into a child. Choose a position c strictly inside the interval of values still possible for X_x. Replace its parent coordinate by the constant c. The inequality imposed by each arc incident to x then tightens an interval bound on its other endpoint:

$$y \to x \quad \Longrightarrow \quad b_y \leftarrow \min(b_y, c), \qquad x \to z \quad \Longrightarrow \quad a_z \leftarrow \max(a_z, c).$$

Retain every other bound already stored in the parent. Remove x and these arcs, then propagate the new bounds through the remaining graph (the operation `Tighten` defined below). Now put x in the child anchored at c, creating that child if necessary. It receives a new coordinate $U_x \in (0,1)$ with no incident arcs. None of the child's previous intervals or arcs changes. We call this operation a *refinement*. Thereafter we may write X_x for its new coordinate, since its old coordinate is no longer a variable.

The fixed parent position specifies where the child block occurs among the other parent items. The new variable specifies where x occurs within that block. Its fresh interval $(0,1)$ allows every placement of x among the child's existing items.

For example, suppose the root contains A, B, C, with the sole requirement $A < B$. If we refine A at $1/2$, then B's parent interval becomes $(1/2, 1)$ and A receives a unit coordinate in the child at $1/2$. If we next refine C at $1/2$, the same child contains two free items, A and C, with no comparison between them. Both orders

$$A < C < B \qquad \text{and} \qquad C < A < B$$

are represented. The comparison $A < B$ has been preserved. The order of A and C remains open. The additional relation $C < B$ was not queried; this example also shows why refinements must be paid for in the proof. The item B cannot enter this child, since $1/2$ is outside its allowed open interval.

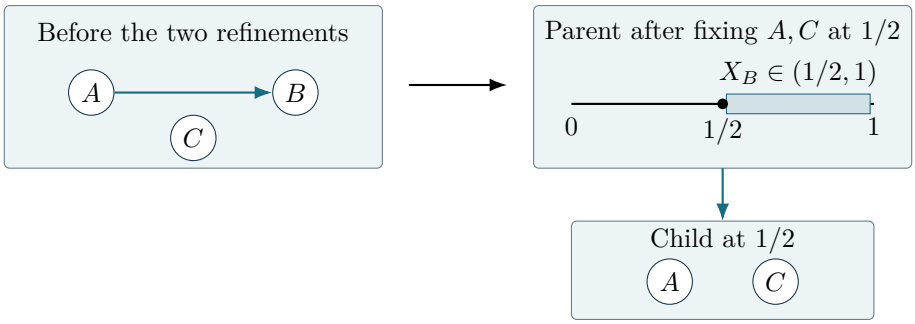


Figure 6: Refining two incomparable items at the same position leaves their mutual order unresolved in a shared child. Their new positions each range over $(0, 1)$ within the child. The parent's bound on B puts the whole child before B.

4.3 Where a comparison is answered

To compare distinct original labels A and B, start at the root. If they belong to the same child subtree, descend into that child. Repeat until their representations differ. At this node, each label is represented either by its own free coordinate or by the anchor of the child containing it. There are three cases.

1. Two different anchors determine the answer from their left-to-right order.
2. For a free coordinate X_x and an anchor c, decide whether $X_x < c$ or $X_x > c$.
3. For two free coordinates, decide whether $X_A < X_B$ or $X_B < X_A$.

In Figure 6, comparing A with C descends into their common child. Comparing either one with B stays at the root and compares $1/2$ with X_B; the stored bound already puts B to the right. More generally, the order of the child anchors determines every comparison between their respective subtrees.

For either case involving free coordinates, let

$$K_v = \{X : a_x < X_x < b_x \text{ for all free } x, \quad X_x < X_y \text{ for every stored arc } x \to y\} \tag{1}$$

be the region of allowed assignments at this node, and let $V_v = \mathrm{vol}(K_v)$. The uniform distribution on K_v gives equal weight to equal-volume parts. For example, the fraction compatible with $A < B$ is

$$p = \mathbb{P}(X_A < X_B) = \frac{\mathrm{vol}(K_v \cap \{X_A < X_B\})}{V_v}.$$

To choose an answer, estimate this ratio. Given $\widehat{p}$ with $|\widehat{p} - p| \le \eta$, choose $A < B$ if $\widehat{p} \ge 1/2$, and $B < A$ otherwise. Either choice retains at least $1/2 - \eta$ of the old volume. The adversary calculates $\widehat{p}$ deterministically from the node's volume constraints.

Store the chosen answer as an arc when both coordinates are free. Each stored arc corresponds to an answered comparison and appears once, including when that comparison is repeated. For a free item x and an anchor c, estimate $\mathbb{P}(X_x < c)$ instead and store the chosen upper or lower interval bound. Translate that direction back to the queried labels, reversing it if x is the second label.

To obtain an estimate, the algorithm keeps a neighborhood of the queried items in the graph and calculates the above ratio exactly in the smaller region. The neighborhood consists of all vertices within a prescribed distance in the underlying undirected graph, together with all arcs between those vertices. Its intervals are the bounds already implied by the full graph. Section 6 specifies the radius and the finite volume calculation, and proves that the resulting estimate is accurate.

4.4 The three reasons for refinement

The estimate just described is available when each vertex has few neighbors, every directed path is short, and at least one queried coordinate has an interval of adequate width. We arrange these conditions as follows. Fix an integer $k \geq 16$ and error $\eta = 1/(100k)$. Use a path threshold H and a width threshold $1/R$, with values given below.

First we include every interval bound implied by the arc inequalities. If $y \to x$ and $X_y > a_y$, then also $X_x > a_y$; an upper bound propagates in the reverse direction. Writing $y \preceq x$ when $y = x$ or a directed path leads from y to x, the resulting interval is

$$\ell_x = \max_{y \preceq x} a_y, \qquad r_x = \min_{y \succeq x} b_y, \qquad W_x = r_x - \ell_x. \tag{2}$$

We call (ℓ_x, r_x) the *effective interval*. The operation `Tighten` sets $a_x \leftarrow \ell_x$ and $b_x \leftarrow r_x$ for every free item x. It is performed after any change to the arcs or intervals, and before a width test or probability calculation. These bounds remain stored even if the arcs that implied them are later removed. Tightening does not discard any allowed assignment.

There are exactly three reasons to refine an item.

1. **A comparison needs a wider coordinate.** Call a coordinate *wide* when $W_x \geq 1/R$, and *narrow* otherwise. A comparison estimate needs one wide free coordinate. If all free coordinates in the requested comparison are narrow, refine one of them before answering. It receives an unconstrained unit coordinate in its child. The reason one wide coordinate is enough is explained in Section 6.
2. **A vertex has too many neighbors.** After an answer, if a vertex has at least k neighbors, refine it. This removes all arcs incident to it from the parent graph.
3. **A directed path is too long.** After the degree condition has been restored, select any directed path with H vertices and refine all its items. Complete this selected list before selecting another path. Repeat until no such path remains.

We use the word *repair* for an operation that restores one of these conditions. A path repair consists of the entire selected list. The analysis charges for each completed path repair as a whole.

4.5 How the fixed position is chosen

Anchors are midpoints obtained by repeatedly halving the unit interval. After m halvings the cells have length 2^{-m}, and their midpoints are

$$c = (j + \tfrac{1}{2})2^{-m}, \qquad 0 \leq j < 2^m. \tag{3}$$

We call m the anchor's *level*. For example, $1/2$ has level 0, and $1/4, 3/4$ have level 1. There are 2^m possible anchors at level m. An anchor has only one level, since the reduced denominator of its midpoint is 2^{m+1}. Thus repeated use of a position also reuses its level. This level measures resolution within one parent interval, independently of that parent's depth in the tree. Every refinement moves an item down exactly one tree edge.

For a wide item, we find a midpoint with a substantial fraction of the allowed assignments on either side. Start with a search cell $[0, 1]$. At its midpoint c, estimate $F_x(c) = \mathbb{P}(X_x < c)$ to error η. If the estimate is below $1/4$, search the right half; if it is above $3/4$, search the left half; otherwise accept c. After a rejected test, update the search cell and test its midpoint. Every test uses the same stored intervals, arcs, and feasible region K_v.

For a narrow item, no probability calculation is needed. Choose the unique cell length $b = 2^{-m}$ satisfying $W_x/8 < b \leq W_x/4$. Among the midpoints of that grid, use the one nearest to $(\ell_x + r_x)/2$, taking the smaller one on a tie. It lies near the center of the effective interval.

Both rules choose an interior point. Section 6 proves the property needed for the lower bound: fixing a level-m point recovers a volume factor whose logarithm is at least $(1 - 1/k)m - 4$.

4.6 The mechanical rules

Every answer is also recorded in a separate graph on the original labels, before restoration or, for an answer between anchors, before returning. This routine update is implicit in the pseudocode. We call the comparison history the *transcript* and the graph the *public comparison graph*. Its arcs are never counted as part of a node's active graph. In Section 9 we use it to finish with a compatible fixed order after a prescribed number of queries.

Resolve choices between eligible items by label order. Select a directed path by the lexicographically first eligible label sequence. Degree counts neighbors regardless of arc direction; the length of a directed path counts its vertices.

Algorithm 2: `Restore`(v): restore the degree and path-length bounds.

Input: The node v that has just answered a query.
Output: The DAG at v has degree below k and every directed path has fewer than H vertices.

```
while the DAG at v has a vertex x of degree at least k do
    Refine(x)
while the DAG at v contains a directed path on H vertices do
    P ← the selected path
    foreach x ∈ P in label order do
        Refine(x)
```

Algorithm 3: `Compare`(A, B): the complete comparison procedure.

Input: Distinct item labels A, B and the current adversary state.
Output: An answer $A < B$ or $B < A$, with the state updated.

```
while true do
    Descend from the root through common children to the first node v where the
     representations differ
    if both labels are represented by child anchors then
        answer ← their left-to-right order
        return answer
    Tighten(v)
    if some queried free coordinate is wide then
        break
    Refine(the selected queried free item)
p̂ ← an estimate of ℙ(A < B) to error η
if p̂ ≥ 1/2 then
    answer ← (A < B)
else
    answer ← (B < A)
Store the arc or interval bound for answer at v
Tighten(v)
Restore(v)
return answer
```

After a preparation refinement, Algorithm 3 searches again from the root because moving an item may change the node where the comparison is answered. There are at most two preparation

refinements: once either queried item has received a fresh wide coordinate, it cannot need preparation again before this answer (Lemma 5.4).

For completeness, the parameters used by these rules are

$$\eta = \frac{1}{100k}, \qquad H = 2^{2k}, \qquad B = (k+1)^{2H+1}, \qquad R = 2^{2kB}. \tag{4}$$

Here B bounds how many other vertices can be joined to one vertex by a directed path in either direction during a repair. Section 5 proves that bound. The value of H makes a long path expensive enough to pay for refining all its items. The value of R makes a narrow interval small enough to pay for refining its item. These two estimates are proved in Sections 6 and 7. Finally, Section 9 chooses k as a slowly growing function of n so that the whole algorithm uses polynomial time.

5 Consistency of the answers and graph bounds

The state describes a family of total orders. We must show that this family never becomes empty and that every order in it respects the answers already given. We must also show that the operations keep the graphs within the bounds required by the probability calculations.

5.1 Preserving the represented orders

Recall how a state represents an order. Choose feasible coordinates in each node, order its free items and child anchors from left to right, and replace each anchor by the order represented inside that child. Each child subtree therefore forms a consecutive block. We exclude equalities between free coordinates, or between a free coordinate and an anchor; these occur on a finite union of sets of volume zero.

We maintain the following invariant:

> Every represented total order respects every comparison already answered.

The state may discard some orders that still respect the answers. Legality requires only that at least one remains. Positive volume in every node will ensure this.

There are two graphs to keep distinct. The public comparison graph has the original labels as vertices and records every answer. A directed path in it is an order relation implied by the transcript. An active graph belongs to one node and contains only its stored comparisons between currently free items. After an active arc is removed, the recursive state continues to enforce its corresponding public answer.

Lemma 5.1 (Effective intervals and sections)**.** *For an active DAG, tightening leaves the feasible region unchanged. The region has positive volume if and only if every effective width is positive. Moreover, fixing $X_x = c$ at any $c \in (\ell_x, r_x)$ leaves a positive-volume section in the remaining coordinates.*

Proof. If $y \neq x$ precedes x on a directed path, every feasible assignment satisfies $X_x > X_y > a_y$; the stored bound $X_x > a_x$ covers $y = x$. Taking the maximum over these bounds gives $X_x > \ell_x$. The upper-bound argument is the reverse. Thus all bounds introduced by (2) already hold, so tightening changes no feasible assignment.

Positive volume requires positive widths. Conversely, suppose all widths are positive. Along an arc $x \to y$, the definitions give $\ell_x \leq \ell_y$ and $r_x \leq r_y$. Choose numbers $0 < \theta_x < 1$ increasing in a topological order, and set

$$X_x = (1 - \theta_x)\ell_x + \theta_x r_x.$$

Increasing θ_x to θ_y strictly increases this value because $r_x > \ell_x$. Replacing its endpoints by the larger endpoints ℓ_y, r_y cannot decrease it. Hence $X_x < X_y$. All coordinates are inside their intervals and all arc inequalities hold strictly, so a sufficiently small open neighborhood of this assignment is feasible.

Now fix $X_x = c$ and delete its coordinate. The inequalities imposed by its incident arcs become upper bounds c at its in-neighbors and lower bounds c at its out-neighbors. Upper bounds propagate only backward along arcs, and lower bounds only forward. Since all previous effective bounds remain stored, the new effective intervals are therefore

$$\begin{cases} (\ell_y, \min(r_y, c)), & y \text{ precedes } x, \\ (\max(\ell_y, c), r_y), & y \text{ succeeds } x, \\ (\ell_y, r_y), & y \text{ is incomparable with } x. \end{cases}$$

The first two cases are disjoint because the graph is acyclic. A predecessor retains positive width: $r_y > \ell_y$ and $c > \ell_x \geq \ell_y$. A successor does too: $r_y > \ell_y$ and $c < r_x \leq r_y$. All other widths stay positive. The criterion just proved, applied to the remaining graph, shows that the section has positive volume. □

Lemma 5.2 (Refinement and child admission)**.** *Suppose the node regions have positive volume and the order invariant holds. Refining x at an interior anchor c preserves both properties, including when the child at c already exists. The public transcript implies no relation between x and any label already in that child subtree.*

Proof. Consider first an existing child at c, and denote its label set by S. Choose a feasible assignment in the parent section $X_x = c$, with the other free coordinates pairwise distinct and different from every child anchor. Such an assignment exists by Lemma 5.1. Also choose feasible assignments in the other nodes.

There is a positive distance from c to every other child anchor and every free parent coordinate in this assignment. All incident inequalities at x are strict. Thus, with everything else fixed, x can be moved slightly to either side of c while keeping the old state feasible. The resulting two represented orders place x immediately before or immediately after the entire block S. Both orders respect the transcript. For each $y \in S$, one has $x < y$ and the other has $y < x$; neither relation can therefore be implied by the public comparison graph.

Now take any order represented after the refinement. Deleting x from the child leaves an assignment of its old coordinates and hence an old child order. Apply the preceding perturbation using this assignment's parent coordinates and old child coordinates. It gives two feasible old orders that agree with the new one on every label except x. The new order inserts x somewhere into their child block. No answered comparison joins x to a label in S, so this insertion violates none of them. Relative to every label outside $S \cup \{x\}$, the whole block occupies the old anchor position. The item x is consequently on the same side of each outside label as in either of the two feasible old orders above. Comparisons between other labels are unchanged. The new order therefore respects the entire transcript.

If there was no child at c, the same argument uses an empty block: the new singleton child places x where its fixed parent coordinate stood. Finally, the remaining parent region is a positive-volume section, and the child receives an independent coordinate in $(0, 1)$. All node regions remain positive. □

5.2 The volume gained by refinement

Let $V_v = \mathrm{vol}(K_v)$ be the volume of node v's feasible region, as in the comparison rule. Define

$$Z = \prod_v V_v. \tag{5}$$

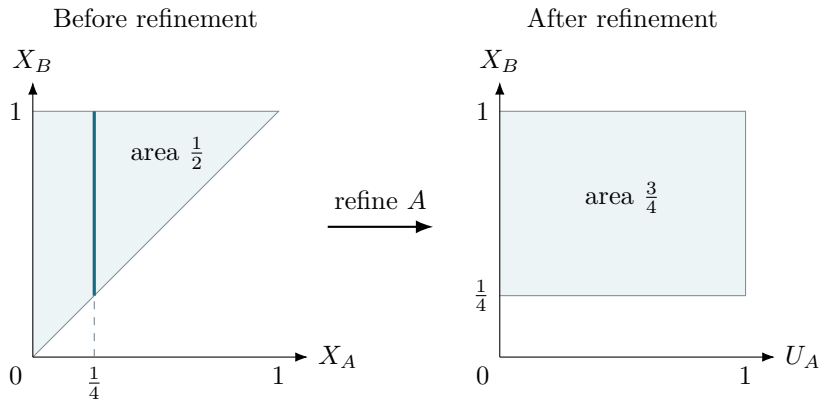


Figure 7: The highlighted section of the triangle has length 3/4. Replacing the fixed parent coordinate by a fresh unit coordinate in the child gives the rectangle on the right. Its area is 3/2 times the original area.

Coordinates in different nodes are separate variables. Their feasible regions form a Cartesian product, so Z is its volume. There is exactly one free coordinate per original item, giving total dimension n. A node with no free coordinates contributes the zero-dimensional volume one. Initially $Z = 1$; whenever all node regions are positive, $0 < Z \leq 1$ because each is contained in its unit cube.

Suppose x is free at a node v with d free coordinates. Write $S_x(c)$ for the $(d-1)$-dimensional volume of the section obtained by fixing $X_x = c$. Slicing K_v along the x coordinate gives $V_v = \int S_x(t)\,dt$. Thus the density of X_x under the uniform volume measure is

$$f_x(c) = \frac{S_x(c)}{V_v}.$$

This density is exactly the factor by which refinement changes Z.

Lemma 5.3 (Refinement identity)**.** *Refining x at an interior anchor c satisfies*

$$\frac{Z_{\text{after}}}{Z_{\text{before}}} = f_x(c). \tag{6}$$

Proof. Let u be the child at c, taking $V_u = 1$ if it is newly created. The parent loses the coordinate X_x and retains precisely the section of volume $S_x(c)$. The child gains a new coordinate $U_x \in (0,1)$, with no constraints involving its old coordinates. Its region becomes $K_u \times (0,1)$ and still has volume V_u. Therefore the affected factors in the product change as follows:

$$V_v V_u \quad \longrightarrow \quad S_x(c) V_u.$$

Their ratio is $S_x(c)/V_v = f_x(c)$. All other node factors stay unchanged. The dimension lost in the parent is restored by the new child coordinate. □

For example, the parent region $0 < X_A < X_B < 1$ has area 1/2. Fixing $X_A = 1/4$ leaves the interval $X_B \in (1/4, 1)$ of length 3/4. The new child coordinate $U_A \in (0,1)$ turns this into a rectangle of area 3/4, so refinement multiplies Z by 3/2; see Figure 7. Both regions represent the order $A < B$. Replacing the fixed parent coordinate by a fresh child coordinate gives the larger area.

5.3 Bounds during a comparison call

We next check how the graph conditions evolve. The numerical lemmas of Section 6 will guarantee interior refinement positions and positive-volume answers whenever the current region is positive

and has the stated graph bounds. We first give the corresponding bounds on graph changes, and then combine the facts in an induction.

Lemma 5.4 (Query preparation)**.** *Before selecting its answer,* COMPARE *performs at most two refinements. Each selected coordinate is narrow, and preparation preserves the graph bounds.*

Proof. A preparation refines one of the two original queried items. Its new coordinate is isolated and has interval $(0, 1)$. Before an answer is stored, any further preparation only removes a parent coordinate and adds an isolated coordinate in a child. It cannot restrict this fresh coordinate. If the query descends into that child, the prepared item is therefore still wide. Each queried item can require preparation at most once. The selection test ensures narrowness. Deleting vertices and admitting isolated vertices preserve the degree and height bounds. □

Lemma 5.5 (Graph bounds and termination)**.** *Suppose a call starts with degree less than k and height less than H, and its selected answer leaves positive volume. Then degree is at most k and height at most $2H$ throughout the call. At its end, every graph again satisfies the strict bounds. Restoration performs at most n refinements, all at the node changed by the answer.*

Proof. Preparation preserves the strict bounds by Lemma 5.4. The answer adds at most one arc, raising any degree by at most one. Any new directed path consists of two old paths joined by that arc, so it has fewer than $2H$ vertices. A positive-volume answer cannot create a directed cycle. An interval change adds no arcs.

Every refinement removes a vertex and its incident arcs from the parent. Thus degrees and heights cannot increase during restoration. The first pass removes every degree violation. The second pass removes every path violation without increasing degrees. All selected path vertices are processed, even if earlier refinements have broken the original path. Each refinement removes a different free item from this node, so there are at most n. A child receives an isolated coordinate and keeps all its old arcs. Its degree and height bounds are preserved as well, so no child requires restoration. □

Consequently a complete call makes at most $n+2$ refinements. During the call, every coordinate has at most $B = (k + 1)^{2H+1}$ other comparable coordinates in its active graph: all lie within undirected distance $2H$ in a graph of degree at most k. These are the bounds used by the local estimator and the anchor rules.

Proposition 5.6 (Legality)**.** *Every reached state has positive volume and represents only orders consistent with the public transcript. After every complete call, all nodes satisfy the strict degree and height bounds.*

Proof. We induct on comparison calls and, within a call, on refinements. Initially the root region is the unit cube, every order is represented, and the graph is empty. Suppose the claims hold at the start of a call.

Preparation refines only narrow coordinates. Its central dyadic rule chooses an interior position, so Lemma 5.2 preserves positivity and the order invariant. Lemma 5.4 shows that preparation terminates with the width needed for a probability query, unless the query has already been decided by two anchors.

The local estimator of Section 6 now applies to the positive queried region with the strict graph bounds. Its chosen answer retains at least $1/2 - \eta > 0$ of the volume and makes that comparison true in every represented order. In particular, a new arc cannot create a directed cycle. An answer between anchors was already true in every represented order. The selected answer is recorded in the public graph before any restoration refinement.

Lemma 5.5 gives degree at most k and height at most $2H$ during restoration. At each refinement these bounds and positivity allow the position guarantees of Section 6 to be used. The

chosen anchor is interior, and Lemma 5.2 preserves the invariant and positivity for the next step. Restoration terminates with the strict bounds. This completes the induction. The numerical results used here concern arbitrary positive-volume regions defined by interval bounds and the inequalities of a DAG, subject to their stated hypotheses; their proofs do not assume legality of this adversary. □

6 Local probabilities and the choice of an anchor

The adversary needs to perform two calculations. To answer a comparison, it estimates the fraction of the current region that supports each answer. To move an item into a child, it finds a position where fixing that item retains enough volume. We describe the calculations first, then state the bounds that justify them.

6.1 Calculating a local probability

Let K be the feasible region of the group handling the query. Its graph may be large. The estimator replaces it temporarily by a small graph around the queried coordinates and calculates the desired probability exactly in that smaller region. For a radius ρ, it performs these steps.

1. Use the effective intervals obtained by tightening the full group's bounds. For a query $X_x < c$, retain the vertices within distance ρ of x in the underlying undirected graph; for $X_x < X_y$, retain the union of these neighborhoods of x and y.
2. Keep every arc whose endpoints are both retained, and keep each retained coordinate's effective interval from the full group. Call the resulting region K_{loc}. This retains interval bounds implied by distant vertices.
3. Calculate the volume V_{loc} of this region. Calculate a second volume N_{loc} after adding the proposed comparison arc or threshold bound to a temporary copy of the local region.
4. Return $\widehat{p} = N_{\mathrm{loc}}/V_{\mathrm{loc}}$.

The denominator is positive because the restriction of a full feasible assignment is feasible locally. These steps leave the group's stored constraints unchanged.

Both volumes are finite exact calculations. List the interval endpoints and enumerate the possible orders of those endpoints and the retained coordinates. Discard an order that violates an interval bound or an inequality imposed by an arc. For each remaining order, the coordinates in a gap between consecutive endpoints occur in a prescribed order. If the gap has length a and contains r coordinates, that part has volume $a^r/r!$. Multiply over the gaps and sum over the accepted orders. Appendix C gives the integer implementation and its cost.

Thus the only approximation is the removal of distant constraints. The following theorem bounds its effect and specifies a sufficient radius. The *degree* of the graph is its maximum number of neighbors, ignoring directions; its *height* is the maximum number of vertices on a directed path. Two vertices are *comparable* if a directed path joins them in either direction.

Theorem 6.1 (Local interval probabilities)**.** *Fix integers $d, h \geq 1$, rational $R \geq 1$, and rational $0 < \varepsilon < 1/4$. There is a computable integer $\rho = \rho(d, h, R, \varepsilon)$ with the following property. Consider a positive-volume region in the unit cube defined by interval bounds and the inequalities of a DAG. Suppose the DAG has degree at most d and height at most h.*

1. *If $W_x \geq 1/R$, the estimator above for $\mathbb{P}(X_x < c)$ has additive error at most ε.*
2. *If at least one of W_x, W_y is at least $1/R$, the estimator above for $\mathbb{P}(X_x < X_y)$ has additive error at most ε.*

All other effective widths may be arbitrarily small and positive. The retained region has at most

$$b = 2(d+1)^{\rho+1}$$

coordinates. With dyadic input bounds, its probabilities can be computed exactly in $(3b)^b \operatorname{poly}(b)$ *arithmetic operations. For a fixed queried coordinate, the same retained region gives the stated accuracy for every threshold* c*.*

The width assumption controls comparisons that are almost tied. The proof shows that the queried positions in the full and retained regions can be made close. A comparison can change only when those positions are close to a tie. More precisely, if x has at most B other comparable labels and y is incomparable with x, then

$$\mathbb{P}(|X_x - X_y| \le a) \le \frac{2(B+1)a}{W_x}.$$

The same bound holds with a fixed threshold in place of X_y. It follows from a bound on the density of x that remains valid after fixing X_y; no independence of the queried positions is required (Corollary A.2). A wide x therefore makes the near-tie event small, regardless of the width of y. If the labels are comparable, their directed path is retained and fixes the answer exactly. This is the reason for preparing a query whose free coordinates are all narrow: moving one of them into a fresh unit interval supplies the needed wide coordinate.

Appendix B proves closeness by comparing a uniform assignment from each region. It repeatedly replaces coordinates by uniform choices from their currently permitted intervals, using the same random numbers in both assignments. Each assignment keeps its original uniform distribution, while the discrepancies at vertices far from the removed constraints decrease. Bounded degree and height give a sufficient number of updates and hence a sufficient radius for the deterministic volume calculation described above.

For the adversary, take $d = k$, $h = 2H$, and $\varepsilon = \eta$. Lemmas 5.5 and 5.4 establish the graph and width bounds at every estimator call.

6.2 Density and the choice of a position

Fix a free coordinate x in a group of volume V. If $S_x(t)$ is the volume of the section obtained by setting $X_x = t$, its density is

$$f_x(t) = \frac{S_x(t)}{V}.$$

It measures volume per unit coordinate length: the fraction of assignments with $a < X_x < b$ is $\int_a^b f_x(t)\,dt$. By Lemma 5.3, fixing x at c and giving it a fresh unit coordinate in a child multiplies Z by $f_x(c)$. We therefore want a large density at the chosen anchor.

The two anchor rules obtain this in different ways. For a wide item we can estimate its cumulative distribution F_x and search for a position with substantial mass on either side. For a narrow item we use its interval endpoints alone. The following density estimates justify both rules.

Suppose x has effective interval (ℓ, r) of width W and at most B other comparable labels. Then

$$f_x(t) \le \frac{B+1}{W}, \qquad f_x(t) \ge \frac{4^{-B}}{W} \quad \left(\ell + \frac{W}{4} \le t \le r - \frac{W}{4}\right). \tag{7}$$

The lower bound gives an endpoint-only choice: every point in the central half has this much density. The upper bound limits the probability in a short interval; it will ensure that the midpoint search cannot continue halving its search interval indefinitely.

For the search we also need a density bound that improves as its cell shrinks. For any $a \le c \le b$,

$$(b-a) f_x(c) \ge \min\{F_x(c), 1 - F_x(c)\}\big(F_x(b) - F_x(a)\big). \tag{8}$$

If both tails at c and the mass of $[a, b]$ are bounded below, this forces the density at c to be a constant times $1/(b-a)$. Appendix A proves (7) and (8) by explicit maps between sections.

6.3 The midpoint search

Recall that the wide-item search starts with $[a, b] = [0, 1]$ and tests its midpoint c. It keeps the right half if the reported cumulative probability is below $1/4$, keeps the left half if it is above $3/4$, and otherwise accepts c. This cell records where to test next. Every test uses the same feasible region K and distribution F_x. The temporary threshold bound supplies the numerator for that test; all tests use the same K_{loc} and denominator.

Lemma 6.2 (Midpoint search)**.** *The midpoint search for a wide item terminates at an interior anchor c. If its accepted cell has length 2^{-m}, then*

$$f_x(c) \geq 2^{m-4}.$$

The search makes at most $\lceil \log(4(B+1)/W_x) \rceil + 1$ tests.

Proof. Every estimate has error at most $\eta < 1/32$. A test reported below $1/4$ has true value below $9/32$, and its midpoint becomes the new left endpoint. A test reported above $3/4$ has true value above $23/32$, and its midpoint becomes the new right endpoint. Since the initial endpoint values are $F_x(0) = 0$ and $F_x(1) = 1$, the search maintains

$$F_x(a) < 9/32, \qquad F_x(b) > 23/32.$$

At acceptance, the report lies in $[1/4, 3/4]$, so

$$\frac{7}{32} \leq F_x(c) \leq \frac{25}{32}, \qquad F_x(b) - F_x(a) \geq \frac{7}{16}.$$

The first bounds put c strictly inside its support. Applying (8) to the accepted cell gives

$$2^{-m} f_x(c) \geq \frac{7}{32}\frac{7}{16} = \frac{49}{512} > \frac{1}{16},$$

which proves the density guarantee.

For termination, the upper density bound in (7) gives, at every cell reached,

$$\frac{7}{16} \leq F_x(b) - F_x(a) \leq \frac{B+1}{W_x}\, 2^{-m}.$$

At $m \geq \lceil \log(4(B+1)/W_x) \rceil$, the upper bound would be at most $1/4 < 7/16$. Starting at $m = 0$ and increasing it by one at each rejection therefore permits at most the stated number of tests. □

6.4 A common bound for both anchor rules

The lower-bound proof will compare the logarithmic volume gain from refinement with the anchor's level m. The wide-item search gives $\log f_x(c) \geq m - 4$. The narrow-item rule gives the weaker bound $m - 3 - 2B$, but the choice $R = 2^{2kB}$ forces $m > 2kB$ whenever that rule is used. Thus its additional loss $2B$ is less than the fraction m/k of the level. Both choices satisfy the following bound.

Lemma 6.3 (Refinement density)**.** *Every anchor chosen by the adversary is interior and satisfies*

$$\log f_x(c) \geq (1 - 1/k)m - 4. \tag{9}$$

For a narrow item its level also satisfies

$$m > 2kB + 2. \tag{10}$$

Proof. For a wide item, Lemma 6.2 gives the stronger lower bound $m-4$. For a narrow item, write $b = 2^{-m}$ for the selected cell length, so $W_x/8 < b \le W_x/4$. The chosen midpoint is at distance at most $b/2 \le W_x/8$ from the interval center. It belongs to the central half and is therefore interior. By (7) and $W_x < 8b$,

$$f_x(c) \ge \frac{4^{-B}}{W_x} > 2^{m-3-2B}.$$

Moreover, $b \le W_x/4 < 1/(4R)$, so $m > \log R + 2 = 2kB + 2$. In particular $2B < m/k$, and hence $m - 3 - 2B > (1-1/k)m - 4$, proving (9). □

The narrow rule must use its prescribed fine grid even if a coarser dyadic midpoint lies inside the interval. Its high level is what allows the weaker central density estimate to satisfy the same bound as the wide-item search.

7 A geometric bound at termination

A correct sorter can finish only when the public answers determine a single order. We want to turn that fact into an upper bound on the volume of the final state. An item's position orders it within the node storing it; its place in the full order also depends on the anchors above that node. Consider a feasible assignment in which A is free at position $3/4$ in the root, while B has position $7/8$ inside a child anchored at $1/4$. The represented order is $B < A$, although the two raw coordinates satisfy $3/4 < 7/8$. Applying the usual $1/n!$ volume bound requires coordinates whose numerical order agrees with the full item order.

We therefore put all items onto one common line. Replace each anchor by a small interval, and place the child's recursively constructed line inside it. The order of items on the resulting line is exactly their represented order. Each item now has a coordinate in the same $(0,1)$, so at termination all resulting assignments lie in one of the $n!$ equal-volume order regions. The only remaining question is how much this placement shrinks volume.

7.1 An interval for each child

Every group may eventually use anchors at arbitrarily fine dyadic levels. At level m there are 2^m possible anchors. Giving each one width proportional to 2^{-m} would use the same total width at every level and would not fit in a bounded interval. We make the widths decrease a little faster, by an extra factor $2^{-m/k}$. This makes their total finite while adding only m/k to the logarithmic cost of a child placement.

Specifically, reserve for each anchor at level m an interval of width

$$w(m) = \frac{2^{-(1+1/k)m}}{4k}. \tag{11}$$

Only children currently present receive intervals. The sum over all possible anchors below is a bound on every such finite set. The factor $4k$ leaves at least half the line for free positions. Indeed, at every node

$$\sum_c w_c \le \frac{1}{4k} \sum_{m \ge 0} 2^{-m/k} = \frac{1}{4k(1-2^{-1/k})} \le \frac{1}{2}. \tag{12}$$

For the last inequality, convexity gives $2^{-t} \le 1 - t/2$ for $0 \le t \le 1$, and we take $t = 1/k$.

At a node, map a free position t distinct from all anchors to

$$G(t) = \frac{t}{2} + \sum_{c<t} w_c. \tag{13}$$

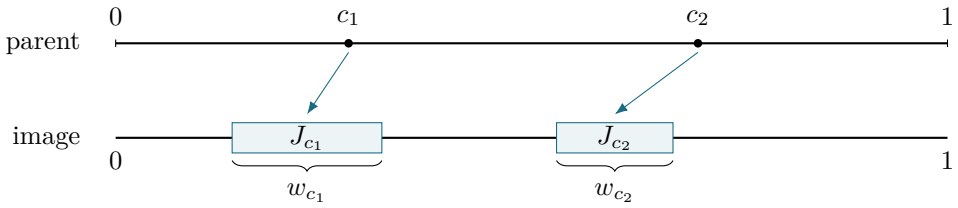


Figure 8: The packing map has slope 1/2 between anchors. Its jump at each anchor reserves an interval for the entire child subtree. The widths are shown schematically; their values are given by (11).

The jump at an anchor c leaves the open interval

$$J_c = \left(\frac{c}{2} + \sum_{a<c} w_a, \frac{c}{2} + \sum_{a<c} w_a + w_c \right).$$

Place the recursively mapped child state inside J_c, using an affine map from $(0,1)$ to J_c. The intervals J_c are disjoint, occur in anchor order, and fit in $(0,1)$ by (12). Free positions stay in the gaps between them. All these widths are used only in the proof.

An item is scaled by a child interval's width every time its route from the root passes through that child. For example, an item at depth two has final position

$$a_1 + w_1\big(a_2 + w_2 G_v(X_x)\big),$$

where a_i is the left endpoint of the chosen child interval, measured in its own parent's unit interval, and G_v is the map in the group where the item is free. Its scale factor is $w_1 w_2/2$. The factor $1/2$ occurs once, at its free coordinate; the ancestor groups contribute only their child widths.

To multiply these scale factors over all items, let s_c be the number of original labels in the subtree below a parent–child edge c. The width w_c scales exactly these s_c items, so its contribution is $w_c^{s_c}$. The product of all child-width contributions is

$$\mathcal{W} = \prod_{\text{parent–child edges } c} w_c^{s_c}. \tag{14}$$

A comparison does not change $\mathcal{W}$. A refinement into a child of width w multiplies it by w: the refined label crosses that edge, while its membership in every other subtree remains unchanged. This holds for both a new child and an existing child.

Lemma 7.1 (Packing volume)**.** *The recursive map is injective and preserves the represented order. Its image $\mathcal{I} \subseteq (0,1)^n$ has volume*

$$\mathrm{vol}(\mathcal{I}) = 2^{-n} Z \mathcal{W}. \tag{15}$$

Consequently, if the transcript determines a unique total order, then

$$n! Z \mathcal{W} \le 2^n. \tag{16}$$

Proof. The map G puts each free position on the correct side of every child interval. Recursion therefore preserves the order of the original labels. Anchors and widths are fixed within the state. The image coordinate of a label depends only on its one free coordinate, through a strictly increasing map, so the whole map is injective.

Partition the domain at the finitely many anchor hyperplanes. On each piece, the map of a label is affine. Its slope is $1/2$ at its free coordinate, multiplied by the widths of the child intervals containing that label along the path to the root. There are exactly n factors of $1/2$, one

per free coordinate. An edge width w_c occurs s_c times. Multiplying these coordinate scale factors shows that every piece has its volume multiplied by $2^{-n}\mathcal{W}$. The domain, a product of the node regions, has volume Z. Injectivity lets us add the image volumes of the pieces, proving (15).

At completion, legality places the image in the region of the cube realizing the unique transcript order. The $n!$ coordinate-order regions have equal volume $1/n!$, since coordinate permutations preserve volume and equality hyperplanes have volume zero. Hence $2^{-n}Z\mathcal{W} \le 1/n!$. □

7.2 Why a long path must use fine anchors

The items on a directed path have already been compared, directly or through intermediate items. Two of them therefore cannot enter the same child. A long path needs many distinct anchors, but there are few anchors at low levels. The following estimate makes this observation quantitative. It will pay for refining an entire path in the next section.

Lemma 7.2 (Total level of a path batch)**.** *The H labels of a selected directed path enter distinct children of their parent. If their levels are $m_1, \dots, m_H$, then*

$$\sum_{i=1}^{H} m_i \ge kH. \tag{17}$$

Proof. Every arc on the selected path records an answered comparison. The path's labels are therefore pairwise comparable in the public transcript, and remain so after arcs are deleted. By Lemma 5.2, two of these labels cannot be admitted to the same child. The procedure completes all H refinements in this parent, so their distinct child widths have sum at most $1/2$. The arithmetic–geometric mean inequality gives

$$\prod_{i=1}^{H} w(m_i) \le (1/(2H))^H.$$

Substitute (11) and take logarithms:

$$(1 + 1/k)\sum_i m_i \ge H\big(\log H - \log(2k)\big).$$

Since $H = 2^{2k}$ and $\log(2k) \le k - 1$ for integers $k \ge 4$, the right-hand side is at least $(k + 1)H$. Division by $1 + 1/k$ proves the claim. The auxiliary inequality follows from $2k \le 2^{k-1}$, true at $k = 4$ and preserved when k increases by one. □

8 The comparison lower bound

The quantity $n!Z\mathcal{W}$ now has the endpoints we need: it starts at $n!$ and is at most 2^n when sorting finishes. If only comparisons could reduce it, and each reduced it by a factor close to 2, the lower bound would follow. Refinements can also reduce it. We modify the quantity so that each complete repair preserves or increases it, leaving comparisons responsible for the required overall decrease.

8.1 Choosing a quantity that repairs cannot decrease

Consider a refinement at level m that removes r incident arcs. The volume identity, the anchor-density bound, and the child width give three simple changes. Write e for the total number of active arcs, and put $L = \log(4k)$.

Quantity	Change caused by the refinement
$\log Z$	at least $(1-1/k)m-4$
$\log \mathcal{W}$	exactly $-(1+1/k)m-L$
e	exactly $-r$

Adding the first two rows gives only $-2m/k-4-L$ as a lower bound. Thus $n!Z\mathcal{W}$ alone does not give the required guarantee. There are two ways to improve it: give more weight to the recovered volume, and reward deletion of arcs.

The repair rules tell us what guarantee to aim for. A narrow-item refinement has $m \geq k$; a degree refinement has $r \geq k$; and a path batch has average level at least k. A lower bound of the form $\tau(m+r-k)$, for some $\tau > 0$, would therefore suffice in all three cases. We seek it using

$$\Psi = n!Z^{\beta}\mathcal{W}\,2^{-\tau e}, \tag{18}$$

with $\beta > 1$. We call Ψ a *potential*: it is a numerical measure of the state for the analysis. The extra power of Z increases the reward for recovered volume. The factor $2^{-\tau e}$ charges a small amount when an arc is stored, then returns that amount when the arc is removed.

The table shows that the change in $\log \Psi$ is at least

$$[\beta(1-1/k)-(1+1/k)]m+\tau r-(4\beta+L).$$

To make this expression equal $\tau(m+r-k)$, choose the coefficients to satisfy

$$\beta(1-1/k)-(1+1/k)=\tau, \qquad 4\beta+L=k\tau. \tag{19}$$

Multiplying the first equation by k and substituting the second gives $\beta(k-5)=k+1+L$. Hence the required values are

$$L=\log(4k), \qquad \beta=\frac{k+1+L}{k-5}, \qquad \tau=\frac{4\beta+L}{k}. \tag{20}$$

For $k \geq 16$ these satisfy $\beta > 1$ and $\tau > 0$. Both modifications preserve the final upper bound: since $Z \leq 1$ and $e \geq 0$, they give $\Psi \leq n!Z\mathcal{W} \leq 2^n$ at termination. Initially they change nothing, because $Z = 1$ and $e = 0$.

Lemma 8.1 (Refinement inequality)**.** *If a refinement selects an anchor at level m for a vertex of current degree r, then*

$$\log \Psi_{\text{new}} - \log \Psi_{\text{old}} \geq \tau(m+r-k). \tag{21}$$

Proof. The child receives an isolated coordinate, so exactly the r parent arcs are deleted. Substituting the three changes in the table into (18) gives the lower bound derived above. The coefficient identities (19) reduce it to $\tau(m+r-k)$. □

8.2 Every complete repair pays for itself

We now apply the same refinement inequality to each of the three repair rules.

Proposition 8.2 (Repair monotonicity)**.** *Each query-preparation refinement and each degree refinement preserves or increases Ψ. Each complete path batch preserves or increases Ψ.*

Proof. A preparation refines a narrow item, so $m > 2kB+2 > k$ by (10). A degree refinement has $r \geq k$. In either case Lemma 8.1 gives a nonnegative change.

For a selected path, let m_i be the level and r_i the current degree at its ith refinement. Summing the refinement inequality over the whole batch gives

$$\log \frac{\Psi_{\text{after}}}{\Psi_{\text{before}}} \geq \tau \left(\sum_{i=1}^{H} m_i + \sum_{i=1}^{H} r_i - kH \right) \geq 0,$$

because Lemma 7.2 gives $\sum_i m_i \geq kH$ and every r_i is nonnegative. □

An individual refinement within a path batch may decrease the potential. The proposition applies to the completed batch, which is why RESTORE finishes a selected path before making another selection or returning an answer.

8.3 Only comparisons can supply the remaining decrease

The endpoint bounds require a net decrease of at least $\log(n!) - n$ in the logarithm of the potential. Complete repairs cannot contribute to this decrease. Each comparison contributes at most $1 + O(\log k/k)$, as we now show. Dividing by this per-comparison bound gives the result.

Theorem 8.3 (Fixed-parameter comparison bound)**.** *For every integer $k \geq 16$, the adversary is deterministic and legal. If a correct sorter terminates after q calls, then, with $a_\eta = -\log(1/2 - \eta)$,*

$$q \geq \frac{\log(n!) - n}{\beta a_\eta + \tau} = \frac{n \log n - O(n)}{1 + O(\log k/k)}. \tag{22}$$

Proof. The procedures and tie rules are deterministic. Legality is Proposition 5.6, and termination of each call follows from Lemmas 5.5 and 5.4.

An answer between free coordinates or between a free coordinate and an anchor retains at least $1/2 - \eta$ of its node volume. It leaves $\mathcal{W}$ unchanged and adds at most one active arc. Its change in $\log \Psi$ is therefore at least $-\beta a_\eta - \tau$. An answer between anchors leaves the potential unchanged. Preparations and complete restorations do not decrease it by Proposition 8.2. Summing over calls yields

$$\log \Psi_{\text{final}} \geq \log(n!) - q(\beta a_\eta + \tau).$$

Initially $Z = \mathcal{W} = 1$ and $e = 0$, explaining the term $\log(n!)$. At termination, Lemma 7.1, $Z \leq 1$, and $e \geq 0$ give

$$\Psi_{\text{final}} = (n! Z \mathcal{W}) Z^{\beta - 1} 2^{-\tau e} \leq 2^n. \tag{23}$$

Combining the last two inequalities proves the first bound.

Finally, (20) gives $\beta = 1 + O(\log k/k)$ and $\tau = O(\log k/k)$. Since $\eta = 1/(100k)$, $a_\eta = 1 + O(1/k)$. The estimate $\log(n!) = n \log n - O(n)$ follows, for example, by comparing $\sum_{j=1}^{n} \log j$ with $\int_1^n \log t \, dt$. These estimates give the second expression in (22). □

9 A uniform polynomial-time implementation

For a fixed parameter k, each local volume calculation is finite and its size is independent of n. To obtain the leading constant one, however, we must let k grow with n. Two costs then need control: computing the parameters themselves, and using them in a comparison. We bound the first by a time limit on initialization and the second by a size limit on the local enumeration. Both limits are in terms of n, the number of items. Computation is measured on a fixed deterministic multitape Turing machine, with integers represented in binary.

9.1 Selecting the parameter

For each k, define

$$\rho_k = \rho(k, 2H, R, \eta), \qquad b_k = 2(k+1)^{\rho_k+1}, \qquad E_k = (3b_k)^{b_k}.$$

Here b_k bounds the number of coordinates in a local region, and E_k bounds the number of orders enumerated to compute its volume. The formula for ρ in Appendix B is computable by integer arithmetic. Fix a program that successively writes complete parameter tuples for $k = 16, 17, \ldots$, including $H, B, R, \rho_k, b_k, E_k$ in ordinary binary notation.

At initialization, run this program for n machine steps. Use only tuples whose entire output has been completed. Among these, choose the largest k satisfying

$$E_k \le n. \tag{24}$$

If there is none, answer every comparison according to label order. The time limit controls the work of producing the parameters even when the next tuple is very large. The program writes only $O(n)$ bits, so checking all completed tuples also takes polynomial time. The size limit (24) ensures that a subsequent local calculation enumerates at most n orders.

These restrictions still allow k to grow without bound. Fix any $k \ge 16$, and let t_k be the finite number of machine steps needed to finish writing its tuple. For every $n \ge \max\{t_k, E_k\}$, that tuple is available and passes the size test. The chosen parameter is then at least k. Since this holds for every fixed k, the selected $k(n)$ tends to infinity.

9.2 Bounding the precision

The enumeration bound counts arithmetic operations. To turn it into a bit bound, we also need to control the binary lengths of all interval endpoints. Each endpoint is 0, 1, or an earlier dyadic anchor. Put

$$K_0 = \lceil \log(4(B+1)R) \rceil + 2.$$

For a wide coordinate, the midpoint-search bound of Lemma 6.2, together with $W_x \ge 1/R$, gives at most K_0 fractional binary digits in the selected anchor. This bound is independent of the precision of the old endpoints. The initialization budget includes writing B and R in full binary notation, so $K_0 = O(n)$.

For a narrow coordinate, suppose the current endpoints have at most Λ fractional digits. They are integer multiples of $2^{-\Lambda}$; a positive effective width is therefore at least $2^{-\Lambda}$. The chosen dyadic cell has length $2^{-m} > W_x/8$, giving $m < \Lambda + 3$. Its midpoint has denominator 2^{m+1} and hence at most $\Lambda + 3$ fractional digits. Thus a narrow refinement adds at most three digits of precision. Tightening takes maxima and minima of existing endpoints and introduces no new ones.

After P refinements, these observations give

$$\Lambda \le K_0 + 3P. \tag{25}$$

By Lemmas 5.4 and 5.5, $P \le (n+2)q$ after q complete calls, with at most $n+2$ further refinements in the next call. Each refinement creates at most one node, so the tree has at most $P + 1$ nodes.

9.3 Bounding the state for every query sequence

A sorter may repeat comparisons indefinitely. To keep the state size and precision bounded independently of the number of repeated questions, after

$$Q_{\text{stop}} = 2n \lceil \log(n+1) \rceil \tag{26}$$

complete calls, choose a total order extending the public comparison graph and answer from it thereafter. Such an order is obtained by topological sorting. The graph records every earlier answer, including those determined by anchors, so every subsequent answer remains consistent with the entire transcript. The recursive state can be discarded and the ranks in this fixed order stored instead.

This cutoff exceeds $n \log n$. A sorter reaching it has already incurred more comparisons than the claimed lower bound, so freezing an order cannot invalidate that bound. Before the cutoff, (25) gives

$$\Lambda = O(n^2 \log(n+1)),$$

and the tree has polynomially many nodes. During a wide-item midpoint search, its tested thresholds also have at most K_0 fractional digits. A common dyadic denominator for stored endpoints and the current threshold therefore has polynomial binary length.

Appendix C shows that a local calculation on b coordinates uses integers of

$$O\big(b(\Lambda + \log(b+1))\big)$$

bits. There are at most n enumerated orders by (24), and $b \le b_k \le n$. Checking each order and forming its contribution requires a fixed polynomial number of integer operations. With standard binary arithmetic, one local calculation consequently has a polynomial bit cost whose exponent does not depend on k.

A wide-item refinement uses at most K_0 such calculations. A narrow-item refinement uses only its endpoints. There are at most $n+2$ refinements per call, and at most one additional estimate to choose the answer. The remaining tasks are polynomial graph computations: propagating interval bounds, finding neighborhoods, and selecting a path. Longest path lengths in a DAG are found in reverse topological order; these lengths let us select the lexicographically first path on H vertices greedily, always choosing the first vertex that admits a long enough continuation. Thus every comparison, including the call at which an order is frozen, has a bit cost bounded by one fixed polynomial in n.

Proof of Theorem 1.1. The bounded parameter selection and finite cutoff define a single deterministic polynomial-time adversary. For all sufficiently large n, it selects a parameter, and $k(n) \to \infty$. If sorting ends before the cutoff, Theorem 8.3 gives $q \ge (1 - o(1))n \log n$. If it reaches the cutoff, it has already made more than $n \log n$ comparisons. Legality holds before the cutoff and is preserved by freezing a compatible total order. This proves the theorem. □

A Coordinate densities

We prove the density bounds used to choose anchors and to turn coordinate approximation into probability approximation. Let V be the set of coordinates, let $N = |V|$, and let $K \subseteq \mathbb{R}^V$ be their feasible region. For a coordinate x and a real number t, write

$$K_x(t) = \{(z_i)_{i \ne x} : z \in K,\ z_x = t\}, \qquad S_x(t) = \operatorname{vol}_{N-1} K_x(t).$$

Here $K_x(t)$ is the $(N-1)$-dimensional section obtained by fixing coordinate x at t. The uniform distribution on K therefore gives x the density

$$f_x(t) = \frac{S_x(t)}{\operatorname{vol}_N K}, \qquad F_x(t) = \int_{-\infty}^{t} f_x(s)\, ds.$$

These formulas define f_x pointwise, and we use this definition throughout. If some coordinates have already been fixed, we use the same definitions in the remaining free coordinates. In dimension zero, the volume of the singleton consisting of the empty assignment is one.

Lemma A.1 (Coordinate density bounds)**.** *Suppose x has at most B other comparable labels and effective interval (ℓ, r) of width W. Its density satisfies*

$$f_x(t) \le \frac{B+1}{W}, \tag{27}$$

$$f_x(t) \ge \frac{4^{-B}}{W} \qquad (\ell + W/4 \le t \le r - W/4). \tag{28}$$

These bounds also hold after fixing any set of labels incomparable with x, for every resulting slice of positive volume.

Proof. We may fix any specified set of coordinates incomparable with x before starting the argument. Measure all volumes in the remaining free coordinates, and abbreviate the section volume by $S(t)$.

First we construct a map from the section at t into the section at a larger value s. Raising x may force its descendants to rise, but no other coordinates need move. We compress $[t, r]$ into $[s, r]$ and leave values above r unchanged. More precisely, for $\ell < t < s < r$, put $\alpha = (r-s)/(r-t)$ and apply

$$\phi(z) = \begin{cases} s + \alpha(z - t), & t \le z \le r, \\ z, & z \ge r \end{cases}$$

to x and all its descendants. Every descendant y has value greater than t. Its effective upper endpoint satisfies $r_y \ge r_x = r$, by the definition of effective intervals. A moved coordinate never decreases; if it starts below r, it stays below r, and if it starts at or above r, it stays fixed. Its lower and upper interval bounds are therefore preserved. The function ϕ is strictly increasing, so it preserves the inequalities imposed by arcs between moved coordinates. Every arc crossing the boundary of this set must enter it: an arc leaving a descendant would lead to another descendant. Only its head coordinate can move, and that coordinate never decreases, so its inequality also remains strict.

This injective map sends the section at t into the section at s. To compare volumes, partition the source section according to which moved coordinates lie below r. On each piece the map is affine and its Jacobian determinant in the free coordinates is α^j, where $j \le B$: the coordinate x is fixed, and at most B other coordinates are moved. Since $0 < \alpha < 1$, this determinant is at least α^B. The images of distinct pieces are disjoint because the map is injective. Adding their volumes gives

$$S(s) \ge \left(\frac{r-s}{r-t}\right)^B S(t) \qquad (t < s). \tag{29}$$

For a smaller target value s, the construction is reversed: lower x and its ancestors, compress $[\ell, t]$ into $[\ell, s]$, and leave values below ℓ fixed. Thus, for $\ell < s < t < r$, use

$$\phi(z) = \begin{cases} z, & z \le \ell, \\ s + \dfrac{s-\ell}{t-\ell}(z - t), & \ell \le z \le t. \end{cases}$$

on x and its ancestors. The effective lower endpoint of every ancestor is at most ℓ. No moved coordinate rises, and one that starts at or above ℓ remains there. Thus all intervals are preserved. The map is strictly increasing on the moved set, so it preserves the inequalities imposed by arcs within that set. Every arc crossing its boundary leaves the set. Only its tail coordinate can move, and that coordinate never increases, so these inequalities also remain strict. Its determinant on each piece is at least $((s-\ell)/(t-\ell))^B$. Consequently

$$S(s) \ge \left(\frac{s-\ell}{t-\ell}\right)^B S(t) \qquad (s < t). \tag{30}$$

Neither construction moves a coordinate incomparable with x, so both remain valid after the conditioning at the start of the proof. A region of positive volume has some section of positive volume. The two maps carry that section to every $t \in (\ell, r)$ with a positive volume factor. Consequently, after fixing incomparable coordinates, we normalize the density of x over the same effective interval (ℓ, r) of width W.

Normalize S to obtain f_x. Integrating the two inequalities on their respective sides of t gives

$$1 \geq f_x(t) \left(\int_\ell^t \left(\frac{s-\ell}{t-\ell} \right)^B ds + \int_t^r \left(\frac{r-s}{r-t} \right)^B ds \right) = \frac{W f_x(t)}{B+1}.$$

This proves the upper bound. For the lower bound, the density integrates to one over an interval of length W, so some $t_0 \in (\ell, r)$ has $f_x(t_0) \geq 1/W$. Fix $s \in [\ell + W/4, r - W/4]$. If $t_0 < s$, the ratio in (29) is at least $(r-s)/W \geq 1/4$; if $s < t_0$, the ratio in (30) is at least $(s-\ell)/W \geq 1/4$. If $s = t_0$, no map is needed. In every case $f_x(s) \geq 4^{-B}/W$. □

Corollary A.2 (A bound for small differences)**.** *If y is incomparable with x, or Y is a fixed number, then for every fixed interval I of length a,*

$$\mathbb{P}(X_x - Y \in I) \leq \frac{(B+1)a}{W}, \tag{31}$$

where $Y = X_y$ in the first case.

Proof. In the first case, condition on X_y. Lemma A.1 bounds the conditional density of x by $(B+1)/W$. Integrating over the translate $I + X_y$ and then averaging over X_y proves the claim. For fixed Y, apply the unconditional density bound. □

Lemma A.3 (Balancing sections)**.** *Let f be a coordinate density, extended by zero outside its effective interval. For $u \leq t \leq v$,*

$$f(u)f(v) \leq f(t)f(u+v-t). \tag{32}$$

Proof. Write $S(s) = \mathrm{vol}_{N-1} K_x(s)$. It suffices to construct a volume-preserving injection

$$K_x(u) \times K_x(v) \longrightarrow K_x(u+v-t) \times K_x(t).$$

Both products are measured in their $2(N-1)$ free coordinates. If $t = u$, $t = v$, or $S(u)S(v) = 0$, the inequality is immediate. Otherwise suppose $u < t < v$, take a pair of feasible assignments p, q with $p_x = u$ and $q_x = v$, and put $\delta = t - u$.

We first translate the second assignment by setting $h_i = q_i - \delta$ for every i. Translation preserves all arc inequalities, so both p and h satisfy them. The intermediate assignment h need not satisfy the interval bounds. We shall swap selected coordinates of p and h, then undo the translation on the second assignment.

Which coordinates must be swapped together? For each i, the two available values are p_i and h_i. Join the endpoints of an arc $i \to j$ by an undirected edge exactly when

$$\max(p_i, h_i) \geq \min(p_j, h_j).$$

An edge means that choosing the two endpoint values independently could violate $X_i < X_j$. If there is no edge, either value at i is strictly less than either value at j, so every choice is safe. Figure 9 illustrates the two cases.

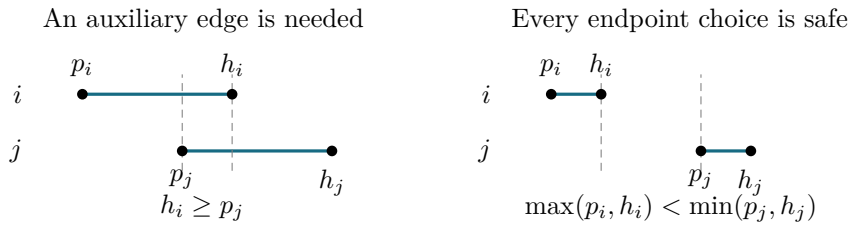


Figure 9: The auxiliary graph in Lemma A.3, for one arc $i \to j$. Each segment joins the two available values of its coordinate. On the left, swapping just one endpoint could select h_i and p_j, violating $X_i < X_j$. On the right, the two segments are separated in the required order, so swapping either endpoint is safe. Every boundary arc of a component has this second form.

Along an edge, $h_i - p_i$ and $h_j - p_j$ have the same strict sign. To see this, if $h_i \geq p_i$ and $h_j \leq p_j$, then $\max(p_i, h_i) = h_i < h_j = \min(p_j, h_j)$, because h satisfies the inequality for the arc $i \to j$. If $h_i \leq p_i$ and $h_j \geq p_j$, the same contradiction follows from $p_i < p_j$. These two cases also exclude a zero difference at either endpoint. Let C be the connected component containing x. Since $h_x - p_x = v - t > 0$, we obtain $h_i > p_i$ for every $i \in C$.

Now define the output pair by

$$(p'_i, q'_i) = \begin{cases} (q_i - \delta,\ p_i + \delta), & i \in C, \\ (p_i, q_i), & i \notin C. \end{cases}$$

This is precisely the proposed swap followed by the inverse translation. At x it gives $p'_x = v - \delta = u + v - t$ and $q'_x = u + \delta = t$, as required.

We check feasibility in two steps. First, on C we have $p_i < q_i - \delta < q_i$ and $p_i < p_i + \delta < q_i$. Thus both new values lie between the original feasible values p_i and q_i, and so remain in the interval for coordinate i. Outside C there is no change. Second, consider the arc inequalities before adding δ back. For an arc with both endpoints in C, the two assignments have exchanged values at both endpoints, so its inequality remains strict in each. If neither endpoint is in C, their values are unchanged. An arc crossing the boundary of C has no corresponding edge in the auxiliary graph, because C is a component; any choice of its endpoint values is therefore safe. Finally, adding the same δ to every coordinate of the second assignment preserves all arc inequalities. Both output assignments are feasible.

To recover the input, subtract δ from q' and observe that for every coordinate

$$\{p'_i, q'_i - \delta\} = \{p_i, q_i - \delta\} = \{p_i, h_i\}.$$

These unordered pairs determine the auxiliary graph. We therefore recover the same component C, swap its coordinates once more, and add δ to the second assignment. This recovers (p, q), proving injectivity.

There are only finitely many possible components C. On the part of the source with a fixed C, the map exchanges selected pairs of free coordinates and adds constants. Its Jacobian determinant has absolute value one. The parts and their images are measurable, and injectivity makes the images disjoint. Adding their volumes proves

$$S(u)S(v) \leq S(t)S(u + v - t).$$

Dividing by the square of the total volume proves the claim. □

Lemma A.4 (Density from mass on both sides)**.** *For a coordinate density f, its cumulative distribution F, and $a \leq c \leq b$,*

$$(b - a)f(c) \geq \min\{F(c), 1 - F(c)\}(F(b) - F(a)). \tag{33}$$

Proof. Lemma A.3 gives $f(u)f(v) \le f(c)f(u+v-c)$ for $u \le c \le v$. For fixed $v \ge c$, integration over $u \le c$ gives

$$F(c)f(v) \le f(c)\int_{-\infty}^{c} f(u+v-c)\,du \le f(c),$$

because the integral of a density over any interval is at most one. Integrating this inequality over $c \le v \le b$ yields

$$F(c)\big(F(b)-F(c)\big) \le (b-c)f(c).$$

Likewise, for fixed $u \le c$, integration over $v \ge c$ gives $f(u)(1-F(c)) \le f(c)$. Integrating over $a \le u \le c$ yields

$$\big(F(c)-F(a)\big)\big(1-F(c)\big) \le (c-a)f(c).$$

Add these inequalities and replace the two tail factors on the left by their minimum. □

B Local approximation of comparison probabilities

We prove Theorem 6.1 by running the same random coordinate updates on two feasible assignments: one in the full region and one in the retained region. Each assignment remains uniformly distributed in its own region. The shared random choices bring their values close together far from the retained boundary. We then use the density bound to show that close values usually give the same comparison answer.

The resulting bound establishes the accuracy of the deterministic volume calculation in Appendix C.

B.1 Coupled coordinate updates

Choose independently an assignment X uniformly from the full feasible region and an assignment $\widehat{X}$ uniformly from the retained region. We shall subsequently update them using shared random numbers. Specifying two random objects in a common experiment in this way is called a *coupling*.

A *sweep* visits every vertex of the full graph once, in a fixed topological order; the retained graph uses the restriction of this order. Consider one assignment and condition on all its coordinates other than X_x. The only constraints still depending on X_x are its interval and the inequalities imposed by arcs incident with x. Its available values form the interval (a, b), where

$$a = \max\{\ell_x,\ X_y : y \to x\}, \qquad b = \min\{r_x,\ X_z : x \to z\}.$$

Because the joint density is constant on the feasible region, the conditional distribution of X_x is uniform on this interval. Replace it by $(1-U_x)a + U_x b$, where U_x is uniform on $(0,1)$. In the retained assignment, use the endpoints computed from $\widehat{X}$ and its retained arcs. Use independent U_x at different vertices and in different sweeps, but share the same U_x between the two assignments whenever x is retained. Vertices outside the retained graph are updated only in the full assignment.

An update replaces a coordinate by a fresh sample from its conditional distribution. For every fixed choice of the other coordinates, this leaves that conditional distribution unchanged; averaging over those choices therefore leaves the distribution of the entire assignment unchanged. Consequently X remains uniform on the full region and $\widehat{X}$ remains uniform on the retained region after every update. They need not remain independent, and no independence between them will be used.

For a retained vertex, write $\Delta_x = |X_x - \widehat{X}_x|$. Within one sweep, superscripts old and new denote values before and after that vertex is visited. Every old value is also its value at the start

of the sweep: updating another vertex does not change it. Thus the whole old discrepancy vector is independent of this sweep's fresh variables U. Whenever every incident arc is retained,

$$\Delta_x^{\text{new}} \le (1-U_x)\max_{y\to x}\Delta_y^{\text{new}} + U_x \max_{x\to z}\Delta_z^{\text{old}}. \tag{34}$$

Here and below an empty maximum is zero. To obtain this inequality, compare the two available lower endpoints: both include the same fixed bound ℓ_x, so their difference is at most the largest discrepancy at an in-neighbor. The same argument for the upper endpoints uses the common bound r_x and the out-neighbors. Multiplying by the shared coefficients $1-U_x$ and U_x gives (34). Topological order explains the superscripts: every in-neighbor has already been visited in this sweep, whereas no out-neighbor has.

We next expand the dependence on the new in-neighbor values until it reaches a fixed interval endpoint. Let $B_h(x)$ denote the vertices at undirected distance at most h from x, suppose this ball is retained, and put

$$D = \max_{y\in B_h(x)} \Delta_y^{\text{old}}.$$

Starting with $z_0 = x$, repeatedly choose an in-neighbor attaining the maximum new discrepancy in (34). Stop at a vertex with no in-neighbor. The resulting reverse directed path is $z_0, z_1, \dots, z_{s-1}$, where $s \le h$ because the graph has height at most h. Write $a_i = 1 - U_{z_i}$.

Every out-neighbor of z_i is at distance at most $i+1 \le h$ from x, so its old discrepancy is at most D. All incident arcs used here are retained for the same reason. Thus (34) gives

$$\Delta_{z_i}^{\text{new}} \le a_i \Delta_{z_{i+1}}^{\text{new}} + (1-a_i)D \quad (0 \le i < s-1), \qquad \Delta_{z_{s-1}}^{\text{new}} \le (1-a_{s-1})D.$$

The last inequality has no in-neighbor term: the lower endpoint at that vertex is fixed and identical in the two regions. Substituting backwards along the path gives

$$\Delta_x^{\text{new}} \le \left(1 - \prod_{i=0}^{s-1} a_i\right) D.$$

Indeed, substituting a bound $(1-P)D$ for the next discrepancy changes the coefficient to $a_i(1-P) + (1-a_i) = 1 - a_i P$.

Let A_x be the minimum of the fresh values $1-U_y$ over x and all its ancestors. Every $a_i \ge A_x$, and $s \le h$ with $0 \le A_x \le 1$, so $\prod_i a_i \ge A_x^s \ge A_x^h$. We have proved

$$\Delta_x^{\text{new}} \le (1 - A_x^h) \max_{y\in B_h(x)} \Delta_y^{\text{old}}. \tag{35}$$

The chosen path may depend on the current discrepancies and the fresh random numbers. Defining A_x using all ancestors makes (35) valid regardless of that choice.

B.2 Moment contraction

The right side of (35) involves up to a bounded number of old discrepancies. We use a sufficiently large power p so that, after taking expectations, its random coefficient compensates for the number of possible old coordinates. The expectation $\mathbb{E}[\Delta_x^p]$ is called the pth moment of the discrepancy. Put

$$M = (d+1)^{h+1}, \qquad a = \frac{1}{4M^2}, \qquad p = (4M^2)^{h+1}.$$

Degree at most d implies that $B_h(x)$ has at most $1 + d + \cdots + d^h \le (d+1)^{h+1} = M$ vertices. Every ancestor of x lies in this ball because any directed path has at most $h-1$ arcs. Thus at most M fresh uniform random variables define A_x. A union bound gives

$$\mathbb{P}(A_x < a) \le Ma = \frac{1}{4M}.$$

On the complementary event, Bernoulli's inequality gives

$$(1 - A_x^h)^p \le (1 - a^h)^p \le \frac{1}{1 + pa^h} = \frac{1}{1 + 4M^2} \le \frac{1}{4M}.$$

Here we used $(1 - z)^{-p} \ge 1 + pz$ for $0 \le z < 1$. On the exceptional event $A_x < a$, the quantity $(1 - A_x^h)^p$ is at most one. Splitting the expectation over these two events therefore gives

$$\mathbb{E}[(1 - A_x^h)^p] \le \mathbb{P}(A_x < a) + \frac{1}{4M} \le \frac{1}{2M}.$$

The fresh random variables defining A_x are independent of all old discrepancies. Raising (35) to the power p and bounding the maximum by a sum therefore yields

$$\begin{aligned} \mathbb{E}[(\Delta_x^{\text{new}})^p] &\le \frac{1}{2M} \sum_{y \in B_h(x)} \mathbb{E}[(\Delta_y^{\text{old}})^p] \\ &\le \frac{1}{2} \max_{y \in B_h(x)} \mathbb{E}[(\Delta_y^{\text{old}})^p]. \end{aligned} \tag{36}$$

B.3 The retained radius

Choose

$$\begin{aligned} C = M + 1, \qquad \delta = \frac{\varepsilon}{16CR}, \\ \lambda = \lceil \log(1/\delta) \rceil, \qquad \mu = \lceil \log(8/\varepsilon) \rceil, \\ t = p\lambda + \mu, \qquad \rho = ht. \end{aligned}$$

The integers λ, μ, t, ρ are computable by rational arithmetic and comparisons with powers of two. The choices ensure $2^\lambda \ge 1/\delta$ and $2^\mu \ge 8/\varepsilon$.

Fix a queried vertex x_*. Its radius-$\rho = ht$ ball is retained. One application of (36) needs information within distance h, so after each sweep we discard an outer layer of that thickness from the set of vertices at which we claim a bound. More precisely, after sweep j consider vertices within distance $h(t - j)$ of x_*. Their h-balls lie within distance $h(t - j + 1)$, where the preceding sweep's bound applies. Initially every discrepancy is at most one, since all coordinates lie in $(0, 1)$. Applying (36) and induction on j now gives

$$\max_{x \in B_{h(t-j)}(x_*)} \mathbb{E}[\Delta_x(j)^p] \le 2^{-j}, \qquad 0 \le j \le t.$$

In particular, Markov's inequality gives

$$\mathbb{P}(\Delta_{x_*}(t) > \delta) \le 2^{-t}\delta^{-p} \le 2^{-\mu} \le \varepsilon/8. \tag{37}$$

For two queried vertices, apply this estimate at each one.

B.4 Comparison probabilities

If the queried labels are comparable, their directed path has at most $h - 1$ arcs and is retained because $\rho \ge h$. The full and local comparison probabilities are then the same, either zero or one.

Otherwise the labels are incomparable. Let x be a queried coordinate of width at least $1/R$. It has at most M other comparable labels, so Corollary A.2 bounds the probability that $X_x - X_y$ lies in any interval of length L by CRL. Outside the two exceptional events in (37), both $|X_x - \widehat{X}_x| \le \delta$ and $|X_y - \widehat{X}_y| \le \delta$. Hence the two differences $X_x - X_y$ and $\widehat{X}_x - \widehat{X}_y$ are within 2δ of each other. They can have different signs only if

$$|X_x - X_y| \le 2\delta.$$

The probability of this event in the full assignment is at most $4CR\delta = \varepsilon/4$. The exceptional events together have probability at most $\varepsilon/4$, so the signs disagree with probability at most $\varepsilon/2$. For a threshold comparison with a fixed number c, there is only one exceptional event. Outside it, different answers require $|X_x - c| \le \delta$, an interval of length 2δ whose probability is at most $2CR\delta = \varepsilon/8$. Including the exceptional event gives a disagreement probability at most $\varepsilon/4$.

For any two events in one random experiment,

$$|\mathbb{P}(E) - \mathbb{P}(\widehat{E})| \le \mathbb{P}(E \text{ and not } \widehat{E}) + \mathbb{P}(\widehat{E} \text{ and not } E).$$

Apply this to the two comparison answers after t sweeps. Since each assignment is still uniform on its original region, these are exactly the full and retained probabilities in the theorem. The bounds just proved are both smaller than the required error ε.

Finally, the retained region has positive volume: the restriction of a full strictly feasible assignment has a small feasible neighborhood. The union of radius-ρ neighborhoods of two vertices has at most $2(d+1)^{\rho+1}$ vertices. This proves the approximation and size claims of Theorem 6.1.

C Exact local volume

Let the retained graph have b coordinates. We compute its volume by partitioning its feasible assignments according to the order of the coordinates and their interval endpoints.

C.1 Enumeration of orders

Sort the distinct interval endpoints as

$$e_1 < e_2 < \cdots < e_s, \qquad s \le 2b.$$

Enumerate the total orders of these endpoints and the b labelled coordinates that preserve the endpoint order. Insert the coordinates successively into the ordered list. The jth insertion has $s + j$ positions, so the number of orders is at most

$$\prod_{j=1}^{b} (s + j) \le (3b)^b.$$

Reject an order if it places a coordinate outside its interval or violates the inequality imposed by an arc. For an accepted order, let r_j count the coordinates between e_j and e_{j+1}. Their relative order is fixed. Permuting these coordinates partitions their cube into $r_j!$ equal-volume regions, apart from equalities of volume zero. The accepted order therefore has volume

$$\prod_{j=1}^{s-1} \frac{(e_{j+1} - e_j)^{r_j}}{r_j!}. \tag{38}$$

An empty gap contributes one. The interval check excludes coordinates outside the extreme endpoints, so $\sum_j r_j = b$.

Almost every feasible assignment determines exactly one accepted order, and every accepted order describes only feasible assignments. Summing (38) thus gives the volume. For $b = 0$, the volume is one.

C.2 Integer arithmetic and probability queries

Let $Q = 2^{\Lambda}$ be a common denominator of the dyadic endpoints, and write $d_j = Q(e_{j+1} - e_j)$. Every summand in (38) has denominator $b!Q^b$ and integer numerator

$$\frac{b!}{\prod_j r_j!} \prod_j d_j^{r_j}. \tag{39}$$

The factorial quotient is an integer because it counts partitions of b labelled positions into groups of sizes r_j. Sum the numerators and divide by $b!Q^b$ once at the end.

All coordinates lie in $(0, 1)$, so the total volume is at most one. The summands are nonnegative; every partial numerator sum is therefore at most $b!Q^b$. The factors used to form one numerator also have products at most this bound, since $d_j \le Q$ and $\sum_j r_j = b$. Thus the arithmetic uses integers of

$$O\big(b(\Lambda + \log(b+1))\big)$$

bits. Checking an order and calculating its contribution require a polynomial number of arithmetic operations in b. The total is $(3b)^b \operatorname{poly}(b)$ operations, proving the remaining claim of Theorem 6.1.

For a comparison numerator, add its arc and repeat the calculation. For a threshold $c \le \ell_x$, the numerator is zero; for $c \ge r_x$, it equals the denominator. Otherwise replace the queried coordinate's upper endpoint by c and repeat. There are still at most $2b$ endpoints. Use a common Q for numerator and denominator; the factors $b!Q^b$ cancel. Inconsistent constraints yield no accepted order and hence volume zero.

If the resulting probability is N/D with nonnegative integers N, D and $D > 0$, the answer test is $2N \ge D$. The midpoint tests use $4N < D$ and $4N > 3D$. The stage gate (24) bounds the number of enumerated orders by n, and the preceding bit bound completes the arithmetic estimate used in Section 9.

References

[1] Mikhail J. Atallah and S. Rao Kosaraju. *An adversary-based lower bound for sorting.* Information Processing Letters 13(2), 55–57, 1981.

[2] Allan Borodin, Leonidas J. Guibas, Nancy A. Lynch, and Andrew C. Yao. *Efficient searching using partial ordering.* Information Processing Letters 12(2), 71–75, 1981. doi:10.1016/0020-0190(81)90005-3.

[3] Dana Richards and Pravin M. Vaidya. *On the distribution of comparisons in sorting algorithms.* BIT 28, 764–774, 1988. Authors' paper.

[4] Gerth Stølting Brodal, Shiva Chaudhuri, and Jaikumar Radhakrishnan. *The randomized complexity of maintaining the minimum.* Nordic Journal of Computing 3(4), 337–351, 1996. Authors' report, BRICS RS-96-40.

[5] Haim Kaplan, Or Zamir, and Uri Zwick. *The amortized cost of finding the minimum.* Proceedings of the Twenty-Sixth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 757–768, 2015. Published online 22 December 2014. doi:10.1137/1.9781611973730.51. Authors' paper.

[6] Haim Kaplan, Or Zamir, and Uri Zwick. *A sort of an adversary.* Proceedings of the Thirtieth Annual ACM–SIAM Symposium on Discrete Algorithms (SODA), 1291–1310, 2019. doi:10.1137/1.9781611975482.79.

[7] Richard P. Stanley. *Two poset polytopes.* Discrete & Computational Geometry 1, 9–23, 1986. doi:10.1007/BF02187680. Author's copy.

[8] Graham Brightwell and Peter Winkler. *Counting linear extensions.* Order 8(3), 225–242, 1991. doi:10.1007/BF00383444.

[9] Martin Dyer, Alan Frieze, and Ravi Kannan. *A random polynomial-time algorithm for approximating the volume of convex bodies.* Journal of the ACM 38(1), 1–17, 1991. doi:10.1145/102782.102783. Authors' manuscript.

[10] Jeff Kahn and Jeong Han Kim. *Entropy and sorting.* Journal of Computer and System Sciences 51(3), 390–399, 1995. doi:10.1006/jcss.1995.1077.